

# Sentiora Database Schema Design Document v1.2

SENTIORA

Database Schema Design Document

Table of Contents

1. Introduction

2. Database Overview

3. High-Level Database Architecture

4. Database Design Principles

5. Entity Relationship Diagrams (ERD)

6. Database Tables

7. Relationships

8. Index Strategy

9. pgvector Design

10. Object Storage Strategy

11. Redis Strategy

12. Security Design

13. Scalability

14. Migration Strategy

15. Example User Flows

16. Sample SQL Queries (Illustrative Only)

17. Future Expansion

18. Best Practices

19. Sequence Diagrams — Key Workflows

20. Module-Level Architecture Diagrams

21. One-Page Schema Summary

22. Table Dependency Map

23. Naming Conventions Cheat Sheet

24. Diagram Legend Reference

25. Recommended Backend Folder Structure

26. Known Limitations and Trade-offs

27. Performance Targets

28. Memory Lifecycle — Capture Through Deletion

29. Common Query Patterns

## SENTIORA

### Database Schema Design Document

#### AI-Powered Digital Memory & Intelligence Platform

| Field            | Value                                                        |
|------------------|--------------------------------------------------------------|
| Project Name     | Sentiora                                                     |
| Document Title   | Database Schema Design Document                              |
| Version          | v1.2                                                         |
| Status           | Draft for Engineering Review                                 |
| Prepared By      | Solution Architecture (in collaboration with Vansh, Founder) |
| Date             | July 28, 2026                                                |
| Related Document | Sentiora PRD v2.1 FINAL                                      |

(frozen)

### Revision History

| Version | Date       | Author       | Description                                                                                                                                                                                                                                                                 |
|---------|------------|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0.1     | —          | Architecture | Initial internal draft                                                                                                                                                                                                                                                      |
| 1.0     | 2026-07-28 | Architecture | First complete release aligned to PRD v2.1 FINAL; MVP tables fully specified, future tables placeheld<br>Added per-module ERD suite, sequence diagrams, module architecture diagrams, schema summary, dependency map, naming cheat sheet, legend, backend folder structure  |
| 1.1     | 2026-07-29 | Architecture | Production handoff pass: redesigned all diagrams with cardinality labels and A4-safe layout; added Engineering Decision callouts, Purpose/Used By per table group, Developer Notes, Known Limitations, Performance Targets, Memory Lifecycle diagram, Common Query Patterns |
| 1.2     | 2026-07-29 | Architecture |                                                                                                                                                                                                                                                                             |

### Table of Contents

- 1. Introduction
- 2. Database Overview
- 3. High-Level Database Architecture
- 4. Database Design Principles
- 5. Entity Relationship Diagram (ERD)

6. Database Tables
7. Relationships
8. Index Strategy
9. pgvector Design
10. Object Storage Strategy
11. Redis Strategy
12. Security Design
13. Scalability
14. Migration Strategy
15. Example User Flows
16. Sample SQL Queries (Illustrative Only)
17. Future Expansion
18. Best Practices
19. Sequence Diagrams — Key Workflows
20. Module-Level Architecture Diagrams
21. One-Page Schema Summary
22. Table Dependency Map
23. Naming Conventions Cheat Sheet
24. Diagram Legend Reference
25. Recommended Backend Folder Structure
26. Known Limitations and Trade-offs
27. Performance Targets
28. Memory Lifecycle — Capture Through Deletion
29. Common Query Patterns

---

# **1. Introduction**

## **1.1 Purpose**

This document defines the complete database schema for Sentiora — covering authentication, the Memory module, the Intelligence (AI/RAG) module, and the groundwork for the future Footprint and Shield modules. It is the single source of truth backend engineers use to implement the PostgreSQL schema, pgvector embeddings layer, Redis caching layer, and Object Storage integration for MVP v0.1 and its immediate successors.

## **1.2 Scope**

In scope:

- Full relational schema for Authentication, Memory, and AI/Intelligence modules (MVP v0.1 and near-term V1.x)
- Placeholder schema definitions for Footprint, Shield, and System/Billing tables so future modules can be added without breaking existing structures
- pgvector embedding design
- Redis and Object Storage integration patterns
- Security, indexing, and scalability guidance

Out of scope: see 1.6.

## **1.3 Audience**

Backend engineers, database administrators, the founding engineer(s) implementing the FastAPI + PostgreSQL backend, and future engineering hires who need onboarding context.

## **1.4 Objectives**

- Provide a normalized, extensible schema that supports the locked MVP scope (Chrome extension, consent-based capture, semantic search, RAG chat, source citations)

- Ensure the schema does not need destructive rework when Footprint and Shield are built in the next phase
- Establish naming, typing, and auditing conventions so every future table is consistent with the first
- Keep the design lean — normalized where it matters, denormalized (JSONB) where flexibility matters more than joins

## 1.5 Assumptions

- MVP v0.1 client is the Chrome Extension only (per PRD); schema anticipates but does not implement Edge/Firefox/Safari/Desktop/Mobile clients
- A single FastAPI monolith talks to a single PostgreSQL instance for MVP; horizontal scaling (read replicas, sharding) is a post-MVP concern
- Embedding model is swappable; schema does not hard-code a vector dimension in a way that blocks migration
- Footprint and Shield tables in this document are **placeholders** — column-level detail will be finalized in a dedicated Technical Specification for those modules, consistent with the PRD's Decisions Register

## 1.6 Out of Scope

- SQL migration scripts / DDL (explicitly excluded per instructions — this is a design document, not implementation)
- Multi-tenant/enterprise schema (see Section 17)
- Billing/subscription business logic (tables placeheld only, Section 6.6)
- Detailed Footprint/Shield risk-scoring algorithms (belongs to a future Shield Technical Spec)

---

# 2. Database Overview

## 2.1 Why PostgreSQL

PostgreSQL is the primary system of record because it gives Sentiora one storage engine that can do three jobs at once:

- **Strict relational integrity** for user, auth, and memory-graph data (foreign keys, constraints, transactions)
- **JSONB support** for semi-structured data (page metadata, AI message content, footprint findings) without needing a separate document store
- **Native vector search via pgvector**, removing the need for a separate vector database at MVP scale

This consolidation matters most for an early-stage team: fewer moving parts, fewer operational failure modes, one backup/restore story.

### ★ Engineering Decision — PostgreSQL over MongoDB.

Sentiora's data is fundamentally relational: a user owns memory items, which belong to collections, which are tagged, which cite chunks, which have embeddings — all of it FK-integrity-dependent. MongoDB would force either (a) denormalized documents that duplicate user/ownership data across collections, breaking on every schema evolution, or (b) manual application-level joins with none of Postgres's transactional guarantees. The one place document-flexibility genuinely helps (per-source metadata) is handled by JSONB columns *inside* Postgres, which gets both worlds without a second database to operate.

## 2.2 Why pgvector

Sentiora Intelligence is fundamentally a RAG system. pgvector lets embeddings live in the same transactional boundary as the content they describe (a `memory_chunk` and its embedding can be inserted atomically), which avoids the consistency problems of syncing a separate vector DB (e.g., Pinecone) with a relational source of truth. At MVP scale (a single user’s personal memory corpus, not a multi-tenant search engine indexing millions of documents per second), pgvector’s IVFFlat/HNSW indexes are sufficient. Migration to a dedicated vector DB is possible later without changing the surrounding relational schema (Section 9.6).

✦ **Engineering Decision — pgvector over an external vector database.** An external vector DB (Pinecone, Weaviate, Qdrant) adds a second system that must stay consistent with Postgres — every memory-item delete, soft-delete/restore, or permission change would need to be mirrored into it, and any drift between the two silently breaks citation accuracy or leaks another user’s data into a search result. At Sentiora’s MVP data volume (per-user corpora, not web-scale indexing), pgvector’s HNSW index gives comparable recall/latency with zero synchronization risk. Revisit only if a single user’s embedding count grows into the tens of millions — not anticipated pre-Series-A.

## 2.3 Why Redis

Redis is not a system of record — it never holds data that isn’t reconstructable from PostgreSQL. It exists purely to make the product feel fast and to protect Postgres:

- Session/JWT blocklist and rate-limiting counters
- Hot caches (recent memory timeline, recent search results)
- Background job queues (capture processing, embedding generation, footprint scans)

✦ **Engineering Decision — Redis as cache, never source of truth.** Every Redis key must be derivable from Postgres. This is a hard rule, not a preference: if a cache flush or Redis outage ever loses data that mattered, that data was in the wrong place. This constrains what Redis can be used for (no “cache-only” features) but makes Redis failures a performance incident, never a data-loss incident.

## 2.4 Why Object Storage

Raw captured artifacts (PDFs, uploaded files, screenshots, exported video) do not belong in PostgreSQL rows. Object Storage (S3-compatible) holds the binary; PostgreSQL holds the pointer (URI), metadata, and everything searchable/structured about it. This keeps the database small, fast to back up, and cheap to scale independently from storage.

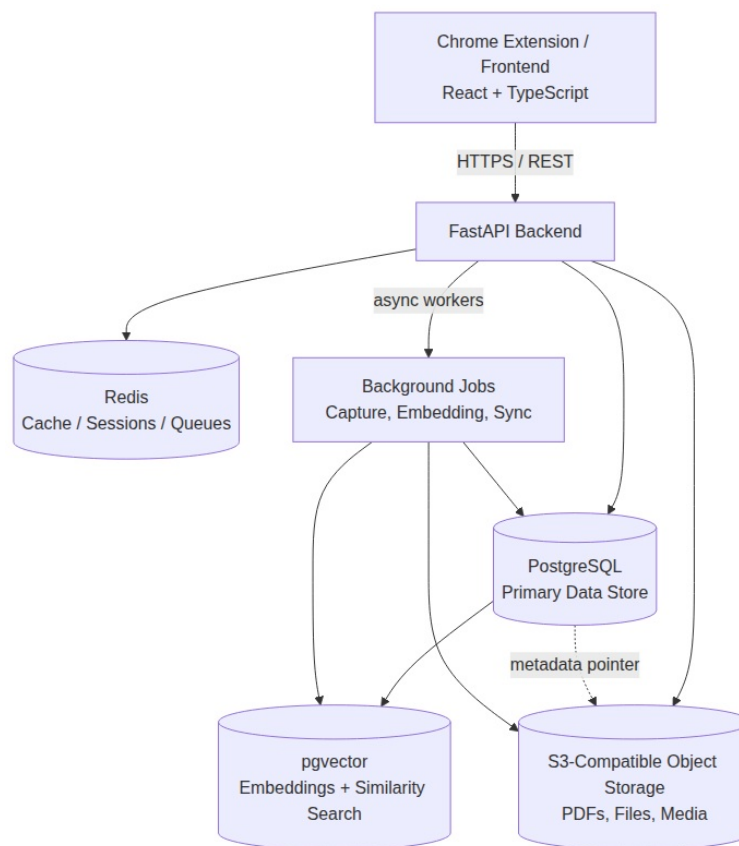
✦ **Engineering Decision — Object Storage over BYTEA/Large Objects.** Storing PDFs as Postgres bytea columns would bloat `pg_dump` backups, slow routine vacuum/backup operations, and pull binary I/O onto the same connection pool serving live user queries. S3-compatible storage separates that I/O path entirely — Postgres only ever stores a URI string.

## 2.5 Overall Storage Architecture

| Layer | Stores               | Does Not Store |
|-------|----------------------|----------------|
|       | Structured entities, |                |

|                                        |                                                                   |                                          |
|----------------------------------------|-------------------------------------------------------------------|------------------------------------------|
| PostgreSQL                             | relationships, JSONB metadata, embeddings (pgvector), audit trail | Large binary files                       |
| pgvector (extension inside PostgreSQL) | Chunk-level embeddings + similarity indexes                       | Raw file bytes                           |
| Redis                                  | Sessions, caches, rate limits, job queues                         | Anything that must survive a cache flush |
| Object Storage (S3-compatible)         | PDFs, uploaded files, exported media, screenshots                 | Searchable/structured data               |

### 3. High-Level Database Architecture



Architecture Flow

#### 3.1 Data Flow Explanation

1. The Chrome Extension captures a user-approved event (page visit, PDF, YouTube video, AI conversation, manual note) and sends it to FastAPI.
2. FastAPI writes the structured record to PostgreSQL (memory\_items + module-specific table) and, if there is a binary asset, uploads it to Object Storage and stores the returned URI on the record.
3. A background job (Section 11.4) chunks the content and requests embeddings, storing vectors in the pgvector-enabled embeddings table linked to memory\_chunks.
4. Redis caches hot paths: the user's session/JWT, the most recent timeline page, and in-flight job status.
5. When the user searches or chats (Sentiora Intelligence), FastAPI

- performs a pgvector similarity query, joins back to `memory_chunks` → `memory_items` for citations, and optionally calls the LLM provider with retrieved context.
6. Every write of consequence is mirrored into `audit_logs` for traceability (Section 12).
- 

## 4. Database Design Principles

**Primary Keys.** Every table uses a UUID (`uuid`, generated via `gen_random_uuid()` at the application or DB level) as its primary key, named `id`. UUIDs avoid leaking sequential counts (e.g., “how many users does Sentiora have”), and they let the client, background workers, and multiple services generate IDs without a round trip to the DB.

✦ **Engineering Decision — UUID over auto-increment integers.** The alternative (`BIGSERIAL`) is marginally smaller and faster to index, but leaks business metrics through IDs and doesn’t support client/offline ID generation (relevant once the Chrome extension needs to create a `memory_items` row before it has confirmed connectivity). The storage/index cost of UUIDv4 is accepted as a deliberate trade-off; if it becomes measurable at scale, UUIDv7 (time-ordered) is a drop-in future upgrade for index locality without changing the column type.

**Foreign Keys.** Declared explicitly, always indexed, always named `<referenced_table_singular>_id` (e.g., `user_id`, `memory_item_id`). Every FK has an explicit `ON DELETE` rule (Section 7.5) — never left to default.

**Soft Delete.** User-facing content tables (`memory_items`, `notes`, `documents`, `collections`, `conversations`) use a `deleted_at` `TIMESTAMPZ` `NULL` column rather than hard deletes. A row with `deleted_at IS NULL` is live. This protects against accidental deletion, supports “trash/restore” UX, and preserves audit continuity. Hard delete is reserved for: (a) a user’s explicit “permanently delete” action after a grace period, and (b) GDPR/account-deletion workflows, which cascade-purge everything including soft-deleted rows.

**Auditability.** Every table includes `created_at` and `updated_at` (`TIMESTAMPZ`, default `now()`, `updated_at` maintained by trigger or application). Sensitive or high-value mutations (`auth`, `permissions`, `security alerts`, `settings changes`) are additionally written to the append-only `audit_logs` table.

**Naming Conventions.** Tables: `snake_case`, plural (`memory_items`, `security_alerts`). Columns: `snake_case`, singular. Booleans prefixed `is/_has_` (`is_active`, `has_consent`). Timestamps suffixed `_at`. Foreign keys suffixed `_id`. No abbreviations that aren’t industry-standard (`id`, `url`, `jwt` are fine; `usr`, `cnv` are not).

**Timestamp Conventions.** All timestamps are `TIMESTAMPZ` (never naive `TIMESTAMP`), stored in UTC, converted to local time only at the presentation layer.

**Normalization.** Core relational structure (`users`, `memory_items`, `conversations`, `alerts`) is normalized to 3NF to avoid update anomalies. Deliberate denormalization is limited to JSONB metadata fields where the shape genuinely varies per row and would otherwise require dozens of nullable columns or an unmanageable number of narrow lookup tables.

**JSONB Usage.** Reserved for: (1) source-specific metadata that differs by capture type (e.g., a YouTube video’s metadata vs. a PDF’s metadata), (2) provider-specific payloads (raw AI provider response),

(3) flexible settings blobs. JSONB is never used for data that needs to be a foreign key, filtered by an indexed relational query, or joined against another table — that data gets a real column instead.

✦ **Engineering Decision — JSONB over EAV tables or per-source columns.** An Entity-Attribute-Value table (generic key/value rows) avoids schema changes but sacrifices type safety and makes every query a self-join. Dozens of nullable per-source columns on `memory_items` avoids that but produces an unmanageably wide table where 90% of columns are null for any given row. JSONB is the pragmatic middle: type-checked at the application layer, indexable via GIN when a field is actually queried (Section 8), and additive — a new source type never requires a migration.

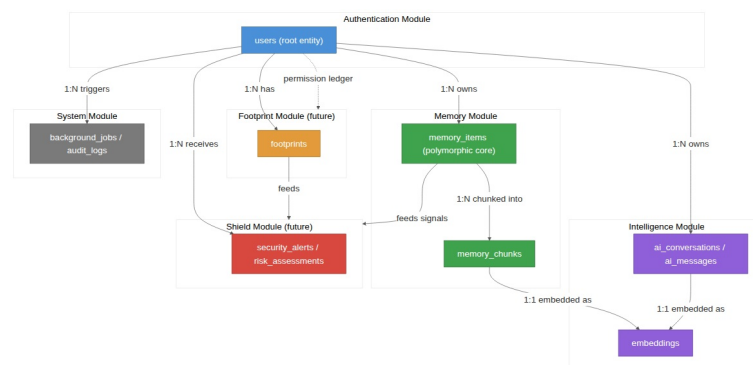
✦ **Developer Note.** Before adding a new JSONB key, check whether it's actually the same key across most rows of that table — if so, it likely wants to be a real column instead (see Best Practice #6, Section 18). A common mistake is letting `memory_items.metadata` become a dumping ground for fields that are actually universal (e.g., language), which then silently escape indexing and validation.

**Scalability Considerations.** Every table anticipated to grow unbounded with usage (memory items, chunks, embeddings, activity logs, audit logs) is designed with partitioning-readiness in mind (Section 13.3) even though MVP does not partition on day one.

## 5. Entity Relationship Diagrams (ERD)

A single all-tables ERD does not stay readable once a schema passes ~15 entities — it becomes noise. This section replaces that with **one overview relationship map** at module granularity, followed by **one detailed ERD per module** (Memory is split in two, since it is the largest module). See Section 24 for the color legend used across every diagram in this document.

### 5.1 Overview: Module Relationship Map



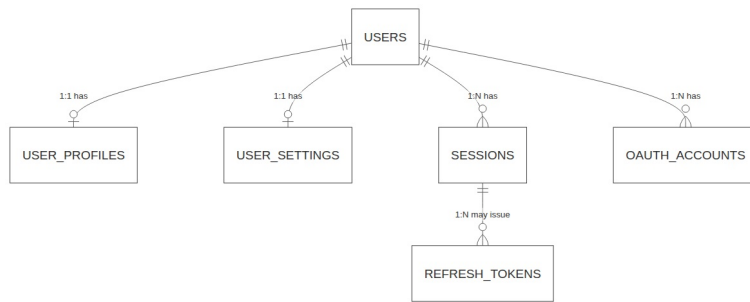
Overview Module Relationship Map

*Read this diagram first.* It shows how the six modules connect through users and the shared `memory_items`/`connected_services` tables, without listing individual columns. Use Sections 5.2–5.7 for entity-level detail within a given module.

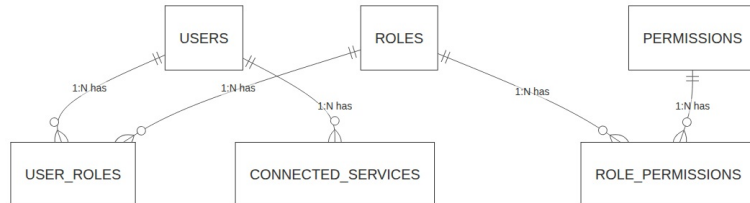
### 5.2 Authentication — Detailed ERD

*Relationship labels use standard cardinality notation (1:1, 1:N, N:N) per Section 24's legend.*



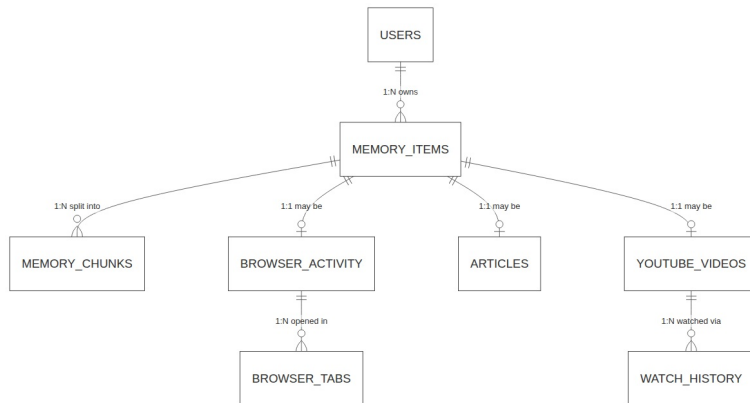


Authentication ERD - Core Identity

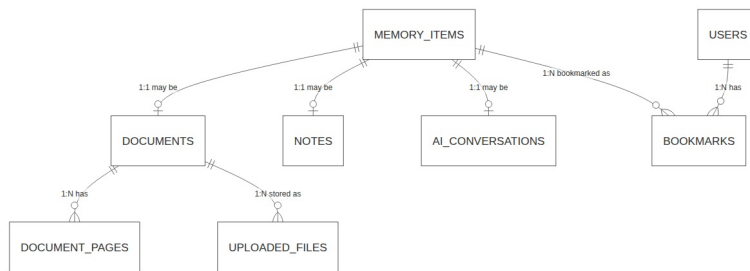


Authentication ERD - RBAC and Permissions

### 5.3 Memory — Detailed ERD (Part 1: Capture & Source Tables)

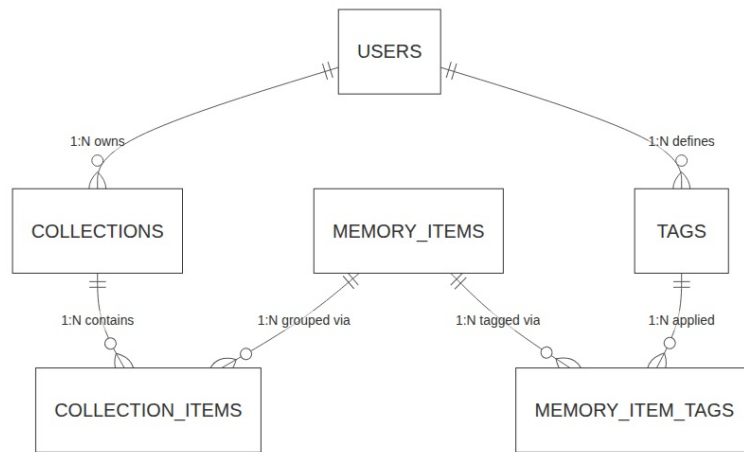


Memory Capture ERD - Browser, Article, YouTube

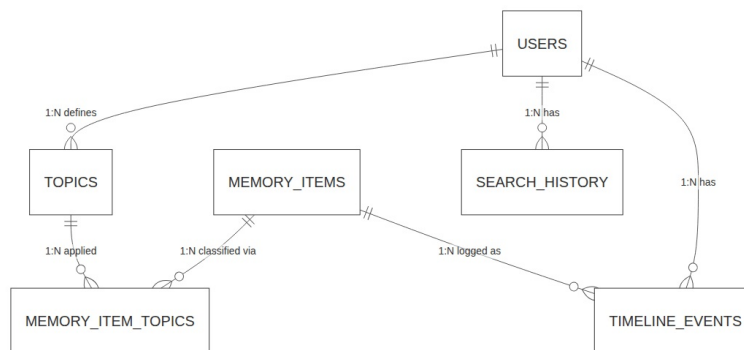


Memory Capture ERD - Documents, Notes, AI, Bookmarks

### 5.4 Memory — Detailed ERD (Part 2: Organization & Discovery Tables)



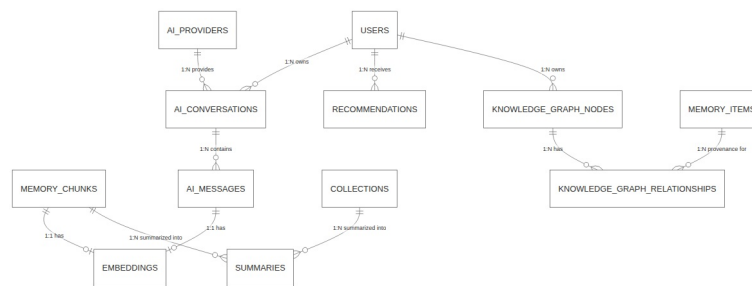
Memory Organization ERD - Collections and Tags



Memory Organization ERD - Topics, Search, Timeline

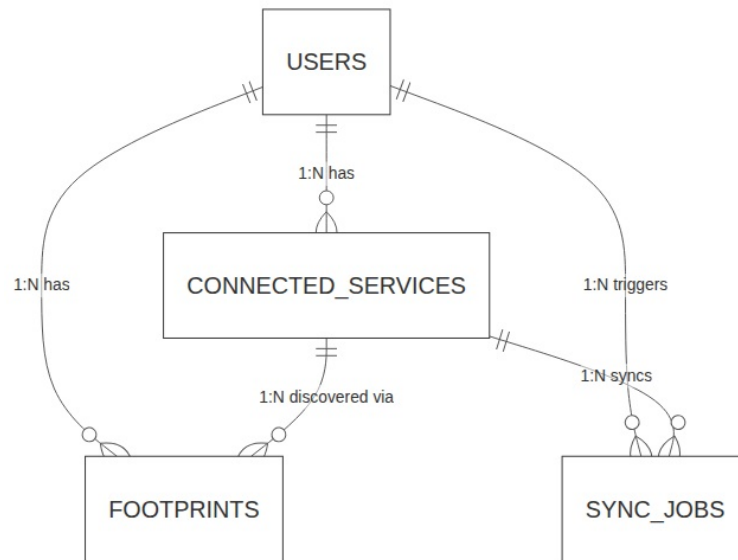
*Memory is split across four diagrams purely for page readability — all four share the same `memory_items/users` root shown in Part 1.*

## 5.5 Intelligence (AI) — Detailed ERD



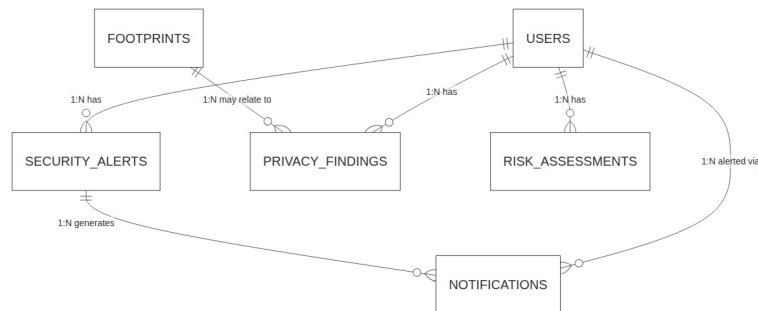
Intelligence ERD

## 5.6 Footprint — Detailed ERD [FUTURE]



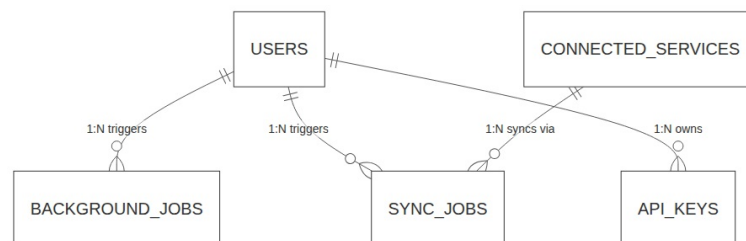
Footprint ERD

## 5.7 Shield — Detailed ERD [FUTURE]

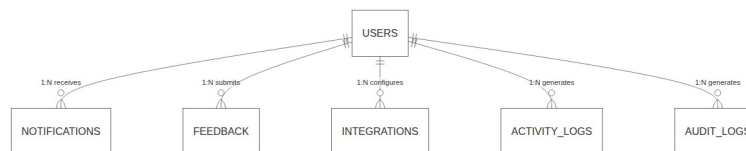


Shield ERD

## 5.8 System — Detailed ERD



System ERD - Jobs and Sync



System ERD - Notifications and Governance

## 6. Database Tables

Each table below documents purpose, columns, constraints, indexes, relationships, notes, and an example record. Tables are grouped exactly as requested. **MVP v0.1 tables are marked [MVP]; near-term V1.x tables are marked [V1.x]; deferred/placeholder tables are marked [FUTURE].**

## 6.1 Authentication

**Purpose.** Establishes identity, session validity, and access control for every other module — nothing in Sentiora executes without resolving to a `users.id` first. **Used By.** Every module (all FKs trace back to `users`); the FastAPI auth middleware on every request; the Chrome extension's login/session flow.

### users [MVP]

**Purpose.** The root identity record for every Sentiora account.

**Description.** One row per registered user. All other user-owned data hangs off this table's `id`.

| Column                         | Type        | Nullable | Default                        |             |
|--------------------------------|-------------|----------|--------------------------------|-------------|
| <code>id</code>                | UUID        | No       | <code>gen_random_uuid()</code> | P           |
| <code>email</code>             | TEXT        | No       | —                              | U<br>r      |
| <code>password_hash</code>     | TEXT        | Yes      | —                              | N<br>o      |
| <code>is_email_verified</code> | BOOLEAN     | No       | <code>false</code>             |             |
| <code>is_active</code>         | BOOLEAN     | No       | <code>true</code>              | A<br>d<br>f |
| <code>last_login_at</code>     | TIMESTAMPTZ | Yes      | —                              |             |
| <code>created_at</code>        | TIMESTAMPTZ | No       | <code>now()</code>             |             |
| <code>updated_at</code>        | TIMESTAMPTZ | No       | <code>now()</code>             |             |
| <code>deleted_at</code>        | TIMESTAMPTZ | Yes      | —                              | S           |

**Constraints.** `UNIQUE(email)`. **PK:** `id`. **FK:** none (root entity).

**Indexes.** Unique index on `email`; partial index on `deleted_at` `IS NULL`.

**Relationships.** One-to-one with `user_profiles`, `user_settings`; one-to-many with everything else in the system.

### Example Record.

```
{ "id": "3f2a...", "email": "vansh@sentiora.app",  
  "is_email_verified": true, "is_active": true }
```

### user\_profiles [MVP]

**Purpose.** Display/profile information kept separate from auth-critical fields.

| Column                    | Type | Nullable | Default                        | No                           |
|---------------------------|------|----------|--------------------------------|------------------------------|
| <code>id</code>           | UUID | No       | <code>gen_random_uuid()</code> | PK                           |
| <code>user_id</code>      | UUID | No       | —                              | FK -<br>user<br>UNI<br>(1:1) |
| <code>display_name</code> | TEXT | Yes      | —                              |                              |
| <code>avatar_url</code>   | TEXT | Yes      | —                              | Poin<br>Obje<br>Stor         |
| <code>bio</code>          | TEXT | Yes      | —                              |                              |
| <code>timezone</code>     | TEXT | No       | <code>'UTC'</code>             |                              |

|            |             |    |         |
|------------|-------------|----|---------|
| locale     | TEXT        | No | 'en-US' |
| created_at | TIMESTAMPTZ | No | now()   |
| updated_at | TIMESTAMPTZ | No | now()   |

**Relationships.** One-to-one with users. **Why separate from users:** auth data changes rarely and is security-sensitive; profile data changes often and is not — separating them keeps the hot auth row small and reduces the blast radius of profile-update bugs.

#### sessions [MVP]

**Purpose.** Tracks active login sessions (device/browser-level), independent of the JWT itself, to support “log out this device” and session auditing.

| Column       | Type        | Nullable | Default           | Notes             |
|--------------|-------------|----------|-------------------|-------------------|
| id           | UUID        | No       | gen_random_uuid() | PK                |
| user_id      | UUID        | No       | —                 | FK → users        |
| device_label | TEXT        | Yes      | —                 | e.g. “iPhone Max” |
| ip_address   | INET        | Yes      | —                 |                   |
| user_agent   | TEXT        | Yes      | —                 |                   |
| is_active    | BOOLEAN     | No       | true              |                   |
| last_seen_at | TIMESTAMPTZ | No       | now()             |                   |
| expires_at   | TIMESTAMPTZ | No       | —                 |                   |
| created_at   | TIMESTAMPTZ | No       | now()             |                   |

**Indexes.** (user\_id, is\_active). **Relationships.** Many-to-one → users. **Notes.** Mirrored in Redis for fast validation (Section 11.2); Postgres is the durable record, Redis is the fast lookup.

#### refresh\_tokens [MVP]

**Purpose.** Long-lived tokens used to mint new JWT access tokens without re-authentication.

| Column     | Type        | Nullable | Default           | Notes                 |
|------------|-------------|----------|-------------------|-----------------------|
| id         | UUID        | No       | gen_random_uuid() | PK                    |
| user_id    | UUID        | No       | —                 | FK → users.id         |
| session_id | UUID        | Yes      | —                 | FK → sessions.id      |
| token_hash | TEXT        | No       | —                 | Never store raw token |
| is_revoked | BOOLEAN     | No       | false             |                       |
| expires_at | TIMESTAMPTZ | No       | —                 |                       |
| created_at | TIMESTAMPTZ | No       | now()             |                       |

**Indexes.** token\_hash (unique). **Notes.** Only the hash is stored — same principle as password storage (Section 12.1).

#### oauth\_accounts [MVP]

**Purpose.** Links a Sentiora user to a third-party identity provider (Google, GitHub) for OAuth login.

| Column                  | Type        | Nullable | Default      |
|-------------------------|-------------|----------|--------------|
| id                      | UUID        | No       | gen_random_u |
| user_id                 | UUID        | No       | —            |
| provider                | TEXT        | No       | —            |
| provider_user_id        | TEXT        | No       | —            |
| access_token_encrypted  | TEXT        | Yes      | —            |
| refresh_token_encrypted | TEXT        | Yes      | —            |
| token_expires_at        | TIMESTAMPTZ | Yes      | —            |
| created_at              | TIMESTAMPTZ | No       | now()        |

**Constraints.** UNIQUE(provider, provider\_user\_id). **Notes.** This table is distinct from connected\_services (below): oauth\_accounts is about **login**, connected\_services is about **data source permissions** feeding Sentiora Memory/Footprint.

#### roles / permissions / role\_permissions / user\_roles [MVP]

**Purpose.** A minimal RBAC model. MVP ships with exactly two roles (user, admin) but the table structure supports growth without redesign.

**roles:** id (UUID, PK), name (TEXT, UNIQUE), description (TEXT), created\_at. **permissions:** id (UUID, PK), key (TEXT, UNIQUE) e.g. memory:read, description (TEXT). **role\_permissions** (bridge): id (UUID, PK), role\_id (FK), permission\_id (FK), UNIQUE(role\_id, permission\_id). **user\_roles** (bridge): id (UUID, PK), user\_id (FK), role\_id (FK), UNIQUE(user\_id, role\_id).

**Notes.** Even though MVP only needs “is this user an admin,” building the bridge-table pattern now avoids a painful migration when team workspaces (Section 6.6) introduce per-workspace roles.

#### user\_settings [MVP]

**Purpose.** Stores per-user preferences and — critically — the **consent/capture toggles** that are core to Sentiora’s trust model.

| Column              | Type  | Nullable |              |
|---------------------|-------|----------|--------------|
| id                  | UUID  | No       | gen_random_u |
| user_id             | UUID  | No       | —            |
| capture_preferences | JSONB | No       | ‘{}’         |

|                          |             |    |                |
|--------------------------|-------------|----|----------------|
| never_capture_categories | JSONB       | No | ["password", " |
| notification_preferences | JSONB       | No | '{'            |
| theme                    | TEXT        | No | 'system'       |
| created_at               | TIMESTAMPTZ | No | now()          |
| updated_at               | TIMESTAMPTZ | No | now()          |

**Why JSONB here specifically.** Capture preferences are a growing, per-source list (today: 6 sources; tomorrow: Slack, Notion, Drive). Modeling each as a column would require a migration per new source. JSONB lets the product add sources without a schema change, while `never_capture_categories` still enforces the PRD's **hard never-capture policy** as an application-layer invariant validated on every write.

#### connected\_services [MVP]

**Purpose.** Represents a user-approved external data source connection (e.g., “GitHub account connected for repo capture”). This is the permission ledger that both Memory (what can be captured) and, later, Footprint (what accounts exist) will read from.

| Column              | Type        | Nullable | Default           |
|---------------------|-------------|----------|-------------------|
| id                  | UUID        | No       | gen_random_uuid() |
| user_id             | UUID        | No       | —                 |
| service_type        | TEXT        | No       | —                 |
| external_account_id | TEXT        | Yes      | —                 |
| scopes_granted      | JSONB       | No       | '[]'              |
| status              | TEXT        | No       | 'active'          |
| connected_at        | TIMESTAMPTZ | No       | now()             |
| revoked_at          | TIMESTAMPTZ | Yes      | —                 |

**Indexes.** (`user_id`, `service_type`). **Notes.** This table is intentionally generic across modules; it is the single place “what is Sentiora allowed to see” is answered, which both simplifies the consent UI and gives Shield (future) an existing feed to build on rather than a new one.

## 6.2 Memory

**Purpose.** Captures and structures every user-approved digital activity into one searchable, polymorphic record set — the core product surface (“Remember what matters”). **Used By.** Sentiora Memory UI (timeline, capture, bookmarks); Sentiora Intelligence (as the retrieval corpus); future Shield (as a signal source).

### memory\_items [MVP]

**Purpose.** The polymorphic backbone of Sentiora Memory. Every captured “thing” — a webpage, a PDF, a YouTube watch, a note, an AI conversation — has exactly one memory\_items row, regardless of its specific type.

| Column           | Type         | Nullable | Default           |
|------------------|--------------|----------|-------------------|
| id               | UUID         | No       | gen_random_uuid() |
| user_id          | UUID         | No       | —                 |
| source_type      | TEXT         | No       | —                 |
| title            | TEXT         | Yes      | —                 |
| url              | TEXT         | Yes      | —                 |
| summary          | TEXT         | Yes      | —                 |
| meaningful_score | NUMERIC(5,2) | Yes      | —                 |
| captured_at      | TIMESTAMPTZ  | No       | now()             |
| occurred_at      | TIMESTAMPTZ  | Yes      | —                 |
| metadata         | JSONB        | No       | '{ }'             |
| is_pinned        | BOOLEAN      | No       | false             |
| created_at       | TIMESTAMPTZ  | No       | now()             |
| updated_at       | TIMESTAMPTZ  | No       | now()             |
| deleted_at       | TIMESTAMPTZ  | Yes      | —                 |

**Indexes.** (user\_id, captured\_at DESC) for timeline; (user\_id, source\_type); GIN on metadata; partial index WHERE deleted\_at IS NULL.

**Relationships.** One row here corresponds to zero-or-one row in the relevant source-specific table (browser\_activity, articles, youtube\_videos, documents, notes, ai\_conversations). One-to-many with memory\_chunks.

**Notes.** memory\_items holds everything **common** across sources (who, when, where, how meaningful); the source-specific tables hold everything **unique** to that source. This is the classic “supertype/subtype” pattern and is what lets Search/RAG/Timeline query one table instead of unioning six.

### Example Record.

```
{ "id": "a1b2...", "user_id": "3f2a...", "source_type": "article",
"title": "Understanding pgvector", "url": "https://...",
"meaningful_score": 87.5, "captured_at": "2026-07-20T10:00:00Z" }
```

### browser\_activity [MVP]

**Purpose.** Subtype table for a captured webpage visit.

| Column             | Type         | Nullable | Default           |
|--------------------|--------------|----------|-------------------|
| id                 | UUID         | No       | gen_random_uuid() |
| memory_item_id     | UUID         | No       | —                 |
| domain             | TEXT         | No       | —                 |
| dwell_time_seconds | INTEGER      | Yes      | —                 |
| scroll_depth_pct   | NUMERIC(5,2) | Yes      | —                 |



|                   |             |     |       |
|-------------------|-------------|-----|-------|
| page_text_excerpt | TEXT        | Yes | —     |
| favicon_url       | TEXT        | Yes | —     |
| created_at        | TIMESTAMPTZ | No  | now() |

**Relationships.** One-to-one with `memory_items`. One-to-many with `browser_tabs`.

#### browser\_tabs [MVP]

**Purpose.** Represents the tab context a browser activity occurred in (tab group/session), useful for reconstructing “what was I researching.”

| Column              | Type        | Nullable | Default           |
|---------------------|-------------|----------|-------------------|
| id                  | UUID        | No       | gen_random_uuid() |
| browser_activity_id | UUID        | No       | —                 |
| tab_group_label     | TEXT        | Yes      | —                 |
| opened_at           | TIMESTAMPTZ | Yes      | —                 |
| closed_at           | TIMESTAMPTZ | Yes      | —                 |
| referrer_url        | TEXT        | Yes      | —                 |

#### articles [MVP]

**Purpose.** Subtype table for long-form article/webpage content captured with full text (as opposed to a bare browser visit).

| Column               | Type        | Nullable | Default           |
|----------------------|-------------|----------|-------------------|
| id                   | UUID        | No       | gen_random_uuid() |
| memory_item_id       | UUID        | No       | —                 |
| author               | TEXT        | Yes      | —                 |
| published_at         | TIMESTAMPTZ | Yes      | —                 |
| reading_time_minutes | INTEGER     | Yes      | —                 |
| full_text            | TEXT        | Yes      | —                 |
| language             | TEXT        | Yes      | —                 |

#### bookmarks [MVP]

**Purpose.** A user-curated pointer to a memory item, distinct from raw capture — represents explicit “save this.”

| Column         | Type | Nullable | Default           |
|----------------|------|----------|-------------------|
| id             | UUID | No       | gen_random_uuid() |
| user_id        | UUID | No       | —                 |
| memory_item_id | UUID | No       | —                 |
| note           | TEXT | Yes      | —                 |

|            |             |    |       |
|------------|-------------|----|-------|
| created_at | TIMESTAMPTZ | No | now() |
|------------|-------------|----|-------|

**Constraints.** UNIQUE(user\_id, memory\_item\_id).

#### youtube\_videos [MVP]

**Purpose.** Subtype table for a captured YouTube video.

| Column           | Type    | Nullable | Default           |                             |
|------------------|---------|----------|-------------------|-----------------------------|
| id               | UUID    | No       | gen_random_uuid() | PK                          |
| memory_item_id   | UUID    | No       | —                 | FK → memo<br>UNIQUE         |
| video_id         | TEXT    | No       | —                 | YouTube<br>ID               |
| channel_name     | TEXT    | Yes      | —                 |                             |
| duration_seconds | INTEGER | Yes      | —                 |                             |
| transcript       | TEXT    | Yes      | —                 | If avail<br>for RA<br>chunk |
| thumbnail_url    | TEXT    | Yes      | —                 |                             |

#### watch\_history [MVP]

**Purpose.** Records individual watch sessions of a YouTube video (a video can be watched, and re-watched, multiple times — the video’s metadata doesn’t change, but each viewing is its own event).

| Column           | Type         | Nullable | Default           |            |
|------------------|--------------|----------|-------------------|------------|
| id               | UUID         | No       | gen_random_uuid() | PK         |
| youtube_video_id | UUID         | No       | —                 | FK → youtu |
| user_id          | UUID         | No       | —                 | FK → user  |
| watched_seconds  | INTEGER      | Yes      | —                 |            |
| completion_pct   | NUMERIC(5,2) | Yes      | —                 |            |
| watched_at       | TIMESTAMPTZ  | No       | now()             |            |

**Relationships.** Many-to-one → youtube\_videos. **Notes.** Separating youtube\_videos (the content) from watch\_history (the event) avoids duplicating video metadata for every re-watch.

#### documents [MVP]

**Purpose.** Subtype table for PDFs and other document-type captures.

| Column         | Type    | Nullable | Default           |                      |
|----------------|---------|----------|-------------------|----------------------|
| id             | UUID    | No       | gen_random_uuid() | PK                   |
| memory_item_id | UUID    | No       | —                 | FK → memor<br>UNIQUE |
| file_type      | TEXT    | No       | —                 | ‘pdf’, ‘d            |
| page_count     | INTEGER | Yes      | —                 |                      |
| author         | TEXT    | Yes      | —                 |                      |

|                 |        |     |   |                    |
|-----------------|--------|-----|---|--------------------|
| storage_uri     | TEXT   | No  | — | Pointer<br>Storage |
| file_size_bytes | BIGINT | Yes | — |                    |

**Relationships.** One-to-many with document\_pages; one-to-one (or one-to-many, if a document can have multiple stored representations) with uploaded\_files.

#### document\_pages [MVP]

**Purpose.** Page-level text extraction for a document, used both for display (jump to page) and RAG chunk provenance.

| Column       | Type        | Nullable | Default           |           |
|--------------|-------------|----------|-------------------|-----------|
| id           | UUID        | No       | gen_random_uuid() | PK        |
| document_id  | UUID        | No       | —                 | FK - docu |
| page_number  | INTEGER     | No       | —                 |           |
| text_content | TEXT        | Yes      | —                 |           |
| created_at   | TIMESTAMPTZ | No       | now()             |           |

**Constraints.** UNIQUE(document\_id, page\_number).

#### uploaded\_files [MVP]

**Purpose.** Generic file-upload metadata table, used by documents but also by any future feature needing raw file storage (avatars, exports).

| Column            | Type        | Nullable | Default           |  |
|-------------------|-------------|----------|-------------------|--|
| id                | UUID        | No       | gen_random_uuid() |  |
| user_id           | UUID        | No       | —                 |  |
| document_id       | UUID        | Yes      | —                 |  |
| original_filename | TEXT        | No       | —                 |  |
| mime_type         | TEXT        | No       | —                 |  |
| storage_uri       | TEXT        | No       | —                 |  |
| checksum_sha256   | TEXT        | Yes      | —                 |  |
| size_bytes        | BIGINT      | No       | —                 |  |
| created_at        | TIMESTAMPTZ | No       | now()             |  |

**Indexes.** checksum\_sha256 (for dedup lookups).

#### notes [MVP]

**Purpose.** Subtype table for manually created notes/links (the “manual save” MVP feature).

| Column         | Type | Nullable | Default           | Note:                        |
|----------------|------|----------|-------------------|------------------------------|
| id             | UUID | No       | gen_random_uuid() | PK                           |
| memory_item_id | UUID | No       | —                 | FK →<br>memory_ite<br>UNIQUE |

|        |      |    |            |
|--------|------|----|------------|
| body   | TEXT | No | —          |
| format | TEXT | No | 'markdown' |

### ai\_conversations [MVP] (also referenced under AI section)

See Section 6.3 — listed once, linked from both Memory and AI groupings since an imported AI chat is both a `memory_item` subtype and the root of `ai_messages`.

### memory\_chunks [MVP]

**Purpose.** The unit of retrieval for RAG. Every `memory_item`'s textual content (article body, transcript, document pages, note body, AI message text) is split into overlapping chunks at ingestion time.

| Column         | Type        | Nullable | Default           |                         |
|----------------|-------------|----------|-------------------|-------------------------|
| id             | UUID        | No       | gen_random_uuid() | P                       |
| memory_item_id | UUID        | No       | —                 | F<br>r                  |
| chunk_index    | INTEGER     | No       | —                 | O<br>p                  |
| content        | TEXT        | No       | —                 | T<br>e                  |
| token_count    | INTEGER     | Yes      | —                 | e<br>O:<br>{<br>1:<br>p |
| source_locator | JSONB       | Yes      | —                 |                         |
| created_at     | TIMESTAMPTZ | No       | now()             |                         |

**Constraints.** `UNIQUE(memory_item_id, chunk_index)`. **Relationships.** Many-to-one → `memory_items`; one-to-one → `embeddings` (Section 9).

**Notes.** `source_locator` is what makes “Source Citations” (a locked MVP Intelligence feature) precise rather than just “this came from item X” — it lets the UI jump to the exact page or timestamp.

### collections [MVP]

**Purpose.** User-defined folders/groupings of memory items (e.g., “Thesis Research”).

| Column      | Type        | Nullable | Default           | Notes        |
|-------------|-------------|----------|-------------------|--------------|
| id          | UUID        | No       | gen_random_uuid() | PK           |
| user_id     | UUID        | No       | —                 | FK → users.i |
| name        | TEXT        | No       | —                 |              |
| description | TEXT        | Yes      | —                 |              |
| created_at  | TIMESTAMPTZ | No       | now()             |              |
| updated_at  | TIMESTAMPTZ | No       | now()             |              |
| deleted_at  | TIMESTAMPTZ | Yes      | —                 |              |

**Bridge table:** collection\_items (id, collection\_id FK, memory\_item\_id FK, added\_at), UNIQUE(collection\_id, memory\_item\_id) — implements the many-to-many between memory\_items and collections.

---

#### tags [MVP]

**Purpose.** Freeform user or AI-assigned labels.

tags: id (PK), user\_id (FK), name (TEXT), created\_at. UNIQUE(user\_id, name). **Bridge table:** memory\_item\_tags (memory\_item\_id FK, tag\_id FK, UNIQUE pair) — many-to-many with memory\_items.

---

#### topics [V1.x]

**Purpose.** AI-derived topic classification (distinct from user-created tags) — supports Topic Extraction.

topics: id (PK), user\_id (FK), name (TEXT), description (TEXT), created\_at. **Bridge table:** memory\_item\_topics (memory\_item\_id FK, topic\_id FK, relevance\_score NUMERIC(5,2)).

---

#### search\_history [MVP]

**Purpose.** Logs semantic searches for recall (“what did I search last week”) and product analytics.

| Column       | Type        | Nullable | Default           | Notes      |
|--------------|-------------|----------|-------------------|------------|
| id           | UUID        | No       | gen_random_uuid() | PK         |
| user_id      | UUID        | No       | —                 | FK → users |
| query_text   | TEXT        | No       | —                 |            |
| result_count | INTEGER     | Yes      | —                 |            |
| searched_at  | TIMESTAMPTZ | No       | now()             |            |

---

**Indexes.** (user\_id, searched\_at DESC).

---

#### timeline\_events [MVP]

**Purpose.** A denormalized, fast-to-query feed powering the Memory Timeline UI, decoupled from having to union six subtype tables on every page load.

| Column         | Type        | Nullable | Default           | Notes    |
|----------------|-------------|----------|-------------------|----------|
| id             | UUID        | No       | gen_random_uuid() | PK       |
| user_id        | UUID        | No       | —                 | FK       |
| memory_item_id | UUID        | No       | —                 | FK       |
| event_type     | TEXT        | No       | —                 | 'c', 'e' |
| event_at       | TIMESTAMPTZ | No       | now()             |          |

---

**Indexes.** (user\_id, event\_at DESC). **Notes.** Populated by application logic/trigger whenever a memory\_items row (or related bookmark/note) is created — a deliberate denormalization to keep the primary timeline query to a single indexed scan.

---

## 6.3 AI

**Purpose.** Turns raw Memory content into retrievable knowledge — semantic search, RAG chat, and (near-term) summaries/knowledge graph (“Understand what you know”). **Used By.** Sentiora Intelligence UI (search bar, chat); background embedding workers; future Shield recommendations.

### ai\_providers [MVP]

**Purpose.** Registry of LLM providers Sentiora integrates with (Anthropic, OpenAI, etc.), so provider-specific config isn’t hard-coded in application code.

| Column        | Type    | Nullable | Default           | Notes       |
|---------------|---------|----------|-------------------|-------------|
| id            | UUID    | No       | gen_random_uuid() | PK          |
| name          | TEXT    | No       | —                 | ‘anthropic’ |
| default_model | TEXT    | Yes      | —                 |             |
| is_active     | BOOLEAN | No       | true              |             |

### ai\_conversations [MVP]

**Purpose.** Root of an imported or in-app AI chat. Also the memory\_items subtype for source\_type = ‘ai\_conversation’.

| Column         | Type        | Nullable | Default           | Notes     |
|----------------|-------------|----------|-------------------|-----------|
| id             | UUID        | No       | gen_random_uuid() | PK        |
| memory_item_id | UUID        | No       | —                 | FK        |
| user_id        | UUID        | No       | —                 | FK        |
| ai_provider_id | UUID        | Yes      | —                 | FK        |
| title          | TEXT        | Yes      | —                 |           |
| imported_from  | TEXT        | Yes      | —                 | ‘c’<br>e’ |
| created_at     | TIMESTAMPTZ | No       | now()             |           |

### ai\_messages [MVP]

**Purpose.** Individual turns within a conversation (both imported chats and Sentiora’s own RAG chat).

| Column             | Type        | Nullable | Default           |
|--------------------|-------------|----------|-------------------|
| id                 | UUID        | No       | gen_random_uuid() |
| ai_conversation_id | UUID        | No       | —                 |
| role               | TEXT        | No       | —                 |
| content            | TEXT        | No       | —                 |
| citations          | JSONB       | Yes      | —                 |
| token_count        | INTEGER     | Yes      | —                 |
| created_at         | TIMESTAMPTZ | No       | now()             |

**Indexes.** (ai\_conversation\_id, created\_at). **Notes.** citations is the concrete implementation of “Source Citations” for RAG chat responses — each assistant message stores exactly which chunks it drew from.

#### embeddings [MVP]

See Section 9 for full design detail.

| Column          | Type         | Nullable | Default           |
|-----------------|--------------|----------|-------------------|
| id              | UUID         | No       | gen_random_uuid() |
| memory_chunk_id | UUID         | Yes      | —                 |
| ai_message_id   | UUID         | Yes      | —                 |
| vector          | VECTOR(1536) | No       | —                 |
| embedding_model | TEXT         | No       | —                 |
| created_at      | TIMESTAMPTZ  | No       | now()             |

**Constraints.** CHECK (memory\_chunk\_id IS NOT NULL OR ai\_message\_id IS NOT NULL) — an embedding must belong to exactly one embeddable thing.

#### knowledge\_graph\_nodes [V1.x]

**Purpose.** Entities extracted from memory content (people, projects, concepts) — foundation for the Knowledge Graph feature.

| Column     | Type        | Nullable | Default              |
|------------|-------------|----------|----------------------|
| id         | UUID        | No       | gen_random_uuid() PK |
| user_id    | UUID        | No       | — FK → users         |
| node_type  | TEXT        | No       | — 'person'           |
| label      | TEXT        | No       | —                    |
| metadata   | JSONB       | No       | '{}'                 |
| created_at | TIMESTAMPTZ | No       | now()                |

#### knowledge\_graph\_relationships [V1.x]

**Purpose.** Directed edges between nodes.

| Column         | Type | Nullable | Default              |
|----------------|------|----------|----------------------|
| id             | UUID | No       | gen_random_uuid() PK |
| source_node_id | UUID | No       | — FK → nodes         |
| target_node_id | UUID | No       | — FK → nodes         |

|                   |             |     |       |        |
|-------------------|-------------|-----|-------|--------|
| relationship_type | TEXT        | No  | —     | €      |
| memory_item_id    | UUID        | Yes | —     | I<br>( |
| created_at        | TIMESTAMPTZ | No  | now() |        |

#### summaries [V1.x]

**Purpose.** Stores generated summaries at the memory-item or collection level. **Explicitly V1.x**, per PRD v2.1's resolution that grounded summaries are out of MVP Intelligence scope (MVP Intelligence = Semantic Search + RAG Chat + Citations only).

| Column         | Type        | Nullable | Default           |        |
|----------------|-------------|----------|-------------------|--------|
| id             | UUID        | No       | gen_random_uuid() | P      |
| memory_item_id | UUID        | Yes      | —                 | F<br>r |
| collection_id  | UUID        | Yes      | —                 | F<br>C |
| summary_text   | TEXT        | No       | —                 |        |
| citations      | JSONB       | Yes      | —                 |        |
| generated_at   | TIMESTAMPTZ | No       | now()             |        |

**Constraints.** CHECK (memory\_item\_id IS NOT NULL OR collection\_id IS NOT NULL).

#### recommendations [V1.x]

**Purpose.** AI-generated suggestions surfaced to the user (e.g., “revisit this topic,” later feeding Shield’s action recommendations too).

| Column              | Type        | Nullable | Default           |  |
|---------------------|-------------|----------|-------------------|--|
| id                  | UUID        | No       | gen_random_uuid() |  |
| user_id             | UUID        | No       | —                 |  |
| recommendation_type | TEXT        | No       | —                 |  |
| payload             | JSONB       | No       | '{'}              |  |
| is_dismissed        | BOOLEAN     | No       | false             |  |
| created_at          | TIMESTAMPTZ | No       | now()             |  |

## 6.4 Security (Footprint & Shield — placeholder tables, [FUTURE] unless noted)

**Purpose.** Maps the user’s external digital presence (Footprint) and turns that plus Memory signals into risk detection and recommendations (Shield) — deferred to next phase per the PRD.

**Used By.** Future Footprint/Shield UI; activity\_logs/audit\_logs (MVP-active) are used by the security/compliance review process today.

Per the PRD’s Decisions Register, Footprint and Shield are **not MVP**. The tables below are included so the schema has a landing spot for this data without requiring a breaking migration later; **column-level detail is intentionally light** and will be finalized in a dedicated Shield/Footprint Technical Specification.



### footprints [FUTURE]

**Purpose.** One row per discovered external account/presence tied to the user's identity (the Footprint module's core entity).

| Column             | Type        | Nullable | Default           |
|--------------------|-------------|----------|-------------------|
| id                 | UUID        | No       | gen_random_uuid() |
| user_id            | UUID        | No       | —                 |
| service_name       | TEXT        | No       | —                 |
| account_identifier | TEXT        | Yes      | —                 |
| discovery_method   | TEXT        | Yes      | —                 |
| status             | TEXT        | No       | 'active'          |
| discovered_at      | TIMESTAMPTZ | No       | now()             |

**Notes.** Deliberately reuses the shape of `connected_services` where possible — Footprint is largely a read-model over consented connections plus external discovery, not an independent capture pipeline (per PRD: “Footprint... explicitly separate from activity/browsing history” and “Shield does not independently collect data”).

### security\_alerts [FUTURE]

id (PK), user\_id (FK), alert\_type (TEXT), severity (TEXT), description (TEXT), is\_resolved (BOOLEAN, default false), detected\_at (TIMESTAMPTZ), resolved\_at (TIMESTAMPTZ, nullable).

### privacy\_findings [FUTURE]

id (PK), user\_id (FK), finding\_type (TEXT), related\_footprint\_id (FK → footprints.id, nullable), details (JSONB), created\_at (TIMESTAMPTZ).

### risk\_assessments [FUTURE]

id (PK), user\_id (FK), overall\_risk\_score (NUMERIC(5,2)), factors (JSONB), assessed\_at (TIMESTAMPTZ).

### activity\_logs [MVP]

**Purpose.** Lightweight, high-volume log of user actions within Sentiora itself (distinct from `audit_logs`, which is security/compliance-focused). Used for product analytics and “recent activity” surfaces.

| Column     | Type        | Nullable | Default              |
|------------|-------------|----------|----------------------|
| id         | UUID        | No       | gen_random_uuid() PK |
| user_id    | UUID        | No       | — FK → u:            |
| action     | TEXT        | No       | — 'search etc.       |
| context    | JSONB       | Yes      | —                    |
| created_at | TIMESTAMPTZ | No       | now()                |

**Notes.** Candidate for partitioning by month once volume grows (Section 13.3).

### audit\_logs [MVP]

**Purpose.** Append-only, tamper-evident record of security-relevant and data-mutating events (login, permission change, consent toggle, account deletion request). See Section 12.7.

| Column       | Type        | Nullable | Default           |                     |
|--------------|-------------|----------|-------------------|---------------------|
| id           | UUID        | No       | gen_random_uuid() | PK                  |
| user_id      | UUID        | Yes      | —                 | FK → (nulla initial |
| actor_type   | TEXT        | No       | 'user'            | 'user'              |
| event_type   | TEXT        | No       | —                 |                     |
| entity_type  | TEXT        | Yes      | —                 |                     |
| entity_id    | UUID        | Yes      | —                 |                     |
| before_state | JSONB       | Yes      | —                 |                     |
| after_state  | JSONB       | Yes      | —                 |                     |
| ip_address   | INET        | Yes      | —                 |                     |
| created_at   | TIMESTAMPTZ | No       | now()             |                     |

**Notes.** No updated\_at/deleted\_at — this table is insert-only by design; nothing is ever modified or deleted from it in normal operation.

## 6.5 System

**Purpose.** Cross-cutting operational tables (jobs, notifications, feature flags, feedback) that no single module owns but every module depends on. **Used By.** Background workers (background\_jobs, sync\_jobs); the notification service; product/ops teams (feedback, usage\_metrics, feature\_flags).

### notifications [MVP]

id (PK), user\_id (FK), type (TEXT), title (TEXT), body (TEXT), is\_read (BOOLEAN, default false), created\_at.

### background\_jobs [MVP]

**Purpose.** Durable record of async work (embedding generation, capture processing) — the Postgres-side complement to the Redis-side queue (Section 11.4).

| Column       | Type        | Nullable | Default           |           |
|--------------|-------------|----------|-------------------|-----------|
| id           | UUID        | No       | gen_random_uuid() | PK        |
| job_type     | TEXT        | No       | —                 | 'gen etc. |
| status       | TEXT        | No       | 'queued'          | queu      |
| payload      | JSONB       | No       | '{'}              |           |
| error        | TEXT        | Yes      | —                 |           |
| attempts     | INTEGER     | No       | 0                 |           |
| created_at   | TIMESTAMPTZ | No       | now()             |           |
| completed_at | TIMESTAMPTZ | Yes      | —                 |           |

### sync\_jobs [MVP]

**Purpose.** Specifically tracks source-sync operations (e.g., “resync GitHub repos”), as distinct from generic background jobs.

id (PK), user\_id (FK), connected\_service\_id (FK → connected\_services.id), status (TEXT), started\_at, finished\_at, items\_synced (INTEGER).

### api\_keys [V1.x]

**Purpose.** Developer/programmatic access keys (not needed for the Chrome-extension-only MVP user flow, but structurally trivial to include now).

id (PK), user\_id (FK), key\_hash (TEXT, UNIQUE), label (TEXT), is\_active (BOOLEAN), last\_used\_at, created\_at.

### feature\_flags [MVP]

id (PK), key (TEXT, UNIQUE), description (TEXT), is\_enabled\_globally (BOOLEAN), rollout\_percentage (NUMERIC).

### usage\_metrics [V1.x]

id (PK), user\_id (FK), metric\_key (TEXT), metric\_value (NUMERIC), period\_start (TIMESTAMPTZ), period\_end (TIMESTAMPTZ).

### feedback [MVP]

id (PK), user\_id (FK), category (TEXT), message (TEXT), created\_at.

### integrations [FUTURE]

**Purpose.** Config for third-party integrations beyond simple OAuth connections (Slack, Notion, Drive — Section 17). id (PK), user\_id (FK), integration\_type (TEXT), config (JSONB), status (TEXT), created\_at.

---

## 6.6 Future Placeholder Tables [FUTURE]

These exist only as named, minimal stubs so that when multi-user/commercial features are built, they attach cleanly to the existing users table rather than requiring a retrofit.

| Table               | Purpose (one-line)                                                                         |
|---------------------|--------------------------------------------------------------------------------------------|
| subscriptions       | Which plan a user/organization is on and its billing cycle                                 |
| billing             | Invoice/payment history, linked to a payment provider customer ID                          |
| team_workspaces     | Shared workspace container for multi-user Sentiora (post-personal-use pivot)               |
| organizations       | Top-level entity for enterprise customers, owns workspaces and billing                     |
| invitations         | Pending invites to join a workspace/organization                                           |
| enterprise_settings | Org-level policy overrides (SSO requirements, data retention policy, capture restrictions) |

**Design note.** None of these are populated or referenced by MVP code paths. They are documented here purely so the Decisions Register has a place to point to, and so organizations/team\_workspaces can later become the tenant boundary described in Section 17.9 without renaming user\_id columns across the whole schema.

---

## 7. Relationships

**One-to-One.** users ↔ user\_profiles, users ↔ user\_settings, memory\_items ↔ each subtype table (articles, documents, etc.), memory\_chunks ↔ embeddings (per chunk). Enforced via a UNIQUE constraint on the foreign key column.

**One-to-Many.** users → memory\_items, memory\_items → memory\_chunks, documents → document\_pages, ai\_conversations → ai\_messages, youtube\_videos → watch\_history.

**Many-to-Many.** memory\_items ↔ collections via collection\_items; memory\_items ↔ tags via memory\_item\_tags; memory\_items ↔ topics via memory\_item\_topics; roles ↔ permissions via role\_permissions; users ↔ roles via user\_roles.

**Bridge Tables.** All many-to-many bridge tables follow the same shape: their own UUID PK (not a composite PK) plus a UNIQUE constraint on the FK pair. A surrogate PK is used even on bridge tables so that, if metadata about the relationship (e.g., “why was this tagged this way”) ever needs to be attached, no primary-key migration is required.

### Cascade Rules.

| Parent            | Child                                                                              | On Parent Delete                                               |
|-------------------|------------------------------------------------------------------------------------|----------------------------------------------------------------|
| users             | user_profiles,<br>user_settings,<br>sessions,<br>refresh_tokens,<br>oauth_accounts | CASCADE                                                        |
| users             | memory_items                                                                       | Soft-delete cascade (application-enforced), not DB CASCADE     |
| memory_items      | memory_chunks, and<br>all subtype tables                                           | CASCADE (hard delete only fires after memory_item hard-delete) |
| memory_chunks     | embeddings                                                                         | CASCADE                                                        |
| documents         | document_pages                                                                     | CASCADE                                                        |
| ai_conversations  | ai_messages                                                                        | CASCADE                                                        |
| collections       | collection_items                                                                   | CASCADE                                                        |
| roles/permissions | role_permissions                                                                   | CASCADE                                                        |

**Delete Rules — rationale.** Auth-adjacent child tables (sessions, oauth\_accounts, etc.) are true hard-CASCADE: there’s no product reason to keep a session after the user is gone. Content tables (memory\_items and everything under it) are **soft-deleted at the application layer first**; the DB-level CASCADE only applies when a genuine hard purge (account deletion, GDPR request, expired trash) happens, ensuring accidental deletes are always recoverable within the grace window.

✳ **Developer Note.** Common mistake: adding a new subtype table under memory\_items and forgetting its ON DELETE CASCADE — this leaves an orphaned row after a hard purge, silently breaking the “account deletion actually deletes everything” guarantee. Always add the new FK to both the migration and the cascade table above (Section 7).

## 8. Index Strategy

**Primary Indexes.** Every table's UUID id is automatically indexed as the PK. Every foreign key column gets an explicit B-tree index — Postgres does not auto-index FKs, and every join in this schema is FK-driven, so this is non-negotiable for MVP performance.

**Composite Indexes.** Built for the query patterns the product actually runs, not speculatively: - `memory_items(user_id, captured_at DESC)` — timeline, the single most-hit query - `memory_items(user_id, source_type)` — filtered browsing (“show me all PDFs”) - `search_history(user_id, searched_at DESC)` - `sessions(user_id, is_active)`

**GIN Indexes.** On JSONB columns that are actually filtered on, not every JSONB column reflexively: `memory_items.metadata`, `user_settings.capture_preferences`. A GIN index on a JSONB column that's only ever read wholesale (e.g., `ai_messages.citations`, which is read per-row, never filtered) would be wasted write overhead — so it's intentionally omitted there.

**Vector Indexes.** HNSW (preferred for `pgvector`  $\geq 0.5$ , better recall/latency tradeoff than IVFFlat at MVP's data volume) on `embeddings.vector`, built with cosine distance (`vector_cosine_ops`) since embedding similarity is typically compared via cosine.

**Search Optimization.** Full-text search (`tsvector` generated column + GIN index) is added on `memory_items.title` and `articles.full_text/notes.body` as a **complement** to semantic search — exact keyword matches (a specific error message, a proper noun) are sometimes better served by full-text than by embeddings, and offering both is cheap once the GIN index exists.

**Performance Strategy.** Index only what a known query path uses; every index has a write-time cost, and an early-stage product with a fast-moving schema benefits more from not over-indexing prematurely than from covering hypothetical queries.

---

## 9. pgvector Design

**9.1 Embedding Storage.** All embeddings — regardless of source (a memory chunk, an AI message) — live in one `embeddings` table, keyed by whichever parent they belong to (Section 6.3). Centralizing them means the similarity-search query and the HNSW index are written once, not once per content type.

**9.2 Chunk Relationships.** `memory_chunks.memory_item_id` → `memory_items` gives every chunk (and therefore every embedding) a path back to its parent item, its owner (`user_id`), and its subtype-specific detail (page number, timestamp) via `source_locator`.

**9.3 Metadata.** Rather than duplicating `user_id` and `source_type` onto the `embeddings` row itself (denormalization for filtering speed), MVP keeps embeddings normalized and joins through `memory_chunks` → `memory_items` at query time. If query latency at scale demands it, a denormalized `user_id` column can be added to `embeddings` later purely as a filter optimization — this is an additive, non-breaking change flagged for the V1.x performance pass.

**9.4 Similarity Search.** Standard pattern: embed the query text, then `ORDER BY vector <=> :query_vector LIMIT :k`, filtered first by `user_id` (via the join) so one user never retrieves another's memory — this filter is the single most important security property of the whole search path (Section 12).

**9.5 Citation Storage.** A RAG chat response's `ai_messages.citations` JSONB stores the specific `memory_chunk_ids` (and their `source_locator`) used to generate that answer — this is what lets the UI say “from

page 4 of X” rather than just “from X.”

**9.6 Future Model Compatibility.** `embeddings.embedding_model` records which model produced a given vector. Because different models produce different-dimension vectors, `VECTOR(1536)` is today’s dimension (matching the current embedding model) — a model change that alters dimension requires either a new column/table (`embeddings_v2`) or an `ALTER` + full re-embed, either of which is a planned, explicit migration rather than a silent corruption risk. Storing `embedding_model` per row means both generations can coexist during a migration window.

✳ **Developer Note.** Common mistake: forgetting to filter by `user_id` *before* the vector similarity `ORDER BY`. Always join through `memory_chunks` → `memory_items` and apply `WHERE mi.user_id = :user_id` ahead of `ORDER BY ... <=> ...` — pgvector’s HNSW index does not enforce tenant isolation for you. Migration consideration: if `embedding_model` ever changes mid-flight, run new-model embedding generation as a background backfill and only cut the search query over once backfill coverage is confirmed complete for the affected user — never assume 100% coverage from `background_jobs.status`.

---

## 10. Object Storage Strategy

**File Uploads / PDFs / Images / Videos.** All binary content is uploaded directly to S3-compatible storage from the backend (never stored as bytes in Postgres). Postgres stores only `storage_uri`, `mime_type`, `size_bytes`, and an optional `checksum_sha256` for integrity/dedup (Section 6.2, `uploaded_files`).

**Metadata.** Structural/descriptive metadata (page count, duration, author) lives in the relevant subtype table (`documents`, `youtube_videos`), not in Object Storage’s own metadata headers — the database must remain the queryable source of truth.

**Retention.** Object Storage objects are retained as long as their owning `memory_items/documents/uploaded_files` row is not hard-deleted. When a hard delete cascades through Postgres, a background job removes the corresponding object — Postgres deletion and object deletion are deliberately decoupled (async) so a slow storage-provider call never blocks a user-facing delete action.

**Versioning.** Not required for MVP (captured files are treated as immutable snapshots); the bucket may still have provider-level versioning enabled purely as an operational safety net, but the application does not expose “file history” as a feature yet.

---

## 11. Redis Strategy

**11.1 Caching.** Frequently-read, cheap-to-regenerate data: the user’s most recent timeline page, collection lists, feature-flag evaluations. Cache keys are namespaced `sentiora:{entity}:{id}` with short TTLs (minutes, not hours) since Memory content changes frequently.

**11.2 Sessions.** The authoritative session record lives in Postgres (`sessions`); Redis holds a mirrored, fast-lookup copy (`session:{token_hash} → user_id`, TTL matching token expiry) so every authenticated request doesn’t require a Postgres round-trip.

**11.3 Rate Limiting.** Sliding-window counters (`ratelimit:{user_id}:{endpoint}`) protect both the API and, more importantly, the LLM-provider spend behind RAG chat and embedding generation.

**11.4 Queues.** Redis-backed queue (e.g., via RQ/Celery-with-Redis-broker) drives the background workers referenced in Section 3: capture processing, chunking, embedding generation, footprint sync (future). Each enqueued job corresponds to a `background_jobs` Postgres row for durability — Redis holds the “work to do right now,” Postgres holds “what happened, ever.”

**11.5 Temporary Storage.** Short-lived state that doesn’t deserve a table: OAuth state/nonce during login flow, in-progress multi-step capture UI state.

---

## 12. Security Design

**12.1 Password Storage.** `users.password_hash` uses a modern adaptive hash (bcrypt or argon2id) with a per-application pepper stored in a secrets manager (not the database). Never reversible, never logged.

**12.2 JWT.** Short-lived access tokens (minutes-to-low-hours) signed with a rotateable secret/key; validated against the Redis-mirrored session on every request so a revoked session invalidates an otherwise-still-valid JWT immediately.

**12.3 OAuth.** Third-party tokens (`oauth_accounts`, `connected_services`) are stored **encrypted at rest** (application-level envelope encryption, not just relying on disk encryption), since these tokens grant access to external accounts — a Postgres dump alone must not be enough to compromise them.

**12.4 RBAC.** Implemented via `roles/permissions/role_permissions/user_roles` (Section 6.1). Every API endpoint’s authorization check resolves to a permission key (memory: `read`, admin: `manage_users`), never a hard-coded role name check, so granting a new role a subset of admin abilities never requires touching endpoint code.

**12.5 Permission Model.** Beyond RBAC, **consent-based data permissions** (`user_settings.capture_preferences`, `connected_services.scopes_granted`) form a second, orthogonal permission layer specific to Sentiora’s trust model — RBAC controls “what can this user do in the product,” consent controls “what is Sentiora allowed to remember about this user.” These are never merged into one table because they answer fundamentally different questions and have different audiences (RBAC: internal/admin; consent: user-facing, must be simple to reason about).

**12.6 Encryption / Sensitive Columns.** Encrypted at the application layer before storage:  
`oauth_accounts.access_token_encrypted/refresh_token_encrypted`,  
`refresh_tokens.token_hash` (hashed, not encrypted — never needs to be decrypted), any future column holding a raw third-party secret.  
`users.password_hash` is hashed, never encrypted (there is no legitimate reason to ever recover a plaintext password).

**12.7 Audit Logging.** `audit_logs` (Section 6.4) captures: login/logout, permission/role changes, consent-toggle changes, security-alert generation, account-deletion requests, and admin actions. It intentionally does **not** log routine content reads (that’s `activity_logs`, a different, non-forensic table) — audit logging is for “can we reconstruct what happened to this account,” not general analytics.

**12.8 Data Retention.** Soft-deleted content (`deleted_at` set) is purged after a defined grace period (e.g., 30 days) by a scheduled job that performs the real hard delete (cascading through Object Storage per

Section 10). `audit_logs` is retained far longer (compliance-driven, e.g., 1-2+ years) since it is the forensic record, and is never subject to the same grace-period purge as user content.

✳ **Developer Note.** Common mistake: encrypting `oauth_accounts` tokens with a key stored in the same database or config file that a Postgres dump would include — the encryption key must live in a separate secrets manager (e.g., AWS KMS, HashiCorp Vault), not application config checked into the same trust boundary as the DB. Migration consideration: rotating the encryption key requires a re-encrypt pass over `oauth_accounts` and any future encrypted columns — plan this as an explicit background job, not a manual one-off script.

---

## 13. Scalability

**Read Replicas.** Not needed at MVP's single-user-heavy, write-light-relative-to-read profile, but the schema makes no assumption that all reads hit a primary — analytics/reporting queries (Section 16) are written to be replica-safe (no reliance on read-your-own-write within the same request beyond normal transaction boundaries).

**Connection Pooling.** FastAPI workers connect through a pooler (PgBouncer or equivalent) rather than opening raw connections per request — essential once background workers (embedding generation, sync jobs) run alongside the API and could otherwise exhaust Postgres's connection limit.

**Partitioning.** Not implemented at MVP launch, but three tables are explicitly designed to be partition-ready: `activity_logs`, `audit_logs`, and `memory_chunks/embeddings` (all append-heavy, unbounded growth). Partitioning key would be `created_at` (monthly range partitions) for logs, and potentially `user_id` (hash) for memory/embeddings if a future multi-tenant deployment demands per-tenant isolation.

**Large Tables.** `memory_items`, `memory_chunks`, `embeddings`, and `activity_logs` are the tables expected to dominate storage/row-count at scale — these are the ones prioritized for the indexing discipline in Section 8 and the partitioning-readiness above.

**Future Sharding.** If Sentiora moves to a multi-tenant/enterprise model (Section 17.9), `user_id` (or a future `organization_id`) is the natural shard key, since almost every query in this schema is already scoped by user. No redesign is needed to introduce sharding later — only an infrastructure change.

**Background Processing.** Anything that isn't needed synchronously for the API response (embedding generation, footprint scans, summary generation) is offloaded to the job queue (Section 11.4) — this keeps API p95 latency independent of AI-provider latency.

---

## 14. Migration Strategy

**Schema Versioning.** Managed via a linear migration tool (e.g., Alembic for FastAPI/SQLAlchemy). Every migration is a single, reviewed, version-controlled file; schema state is always derivable by replaying migrations from zero.

**Migration Naming.** `YYYYMMDDHHMM_short_description.py` (timestamp-prefixed ensures correct ordering across contributors without merge conflicts on a sequence number).



**Rollback Strategy.** Every migration that can be reversed cleanly (additive changes: new table, new nullable column) includes a `downgrade()`. Destructive migrations (dropping a column/table) are split into two migrations across two releases: first stop writing to/reading from the column (release N), then drop it (release N+1) — this avoids a rollback of release N needing to resurrect already-deleted data.

**Seed Data.** A dedicated seed script populates roles, permissions, `role_permissions`, `ai_providers`, and default `feature_flags` — anything the application assumes exists on a fresh database. Seed data is idempotent (upsert-based) so it can safely re-run in any environment.

**Development vs. Production Migrations.** Development databases may be freely reset/reseeded. Production migrations always run through a staging environment first, are backward-compatible with the currently-deployed application version (no migration ships that a not-yet-updated app instance would crash on), and additive/destructive migrations are never combined in one file.

---

## 15. Example User Flows

**User Signs Up.** Insert users → insert `user_profiles` (1:1) → insert `user_settings` (defaults incl. `never_capture_categories`) → assign default role via `user_roles` → `audit_logs` entry (`event_type = 'user_signup'`).

**Browser Page Is Visited.** Extension sends event → check `user_settings.capture_preferences` for browser capture consent and dwell-time/meaningfulness threshold → if it passes the Meaningful Capture Engine → insert `memory_items` (`source_type='browser'`) → insert `browser_activity` (1:1) → insert `browser_tabs` if applicable → insert `timeline_events` → enqueue `background_jobs` (`chunk_and_embed`).

**YouTube Video Is Watched.** Look up or insert `youtube_videos` (dedup on `video_id`) → if new, insert accompanying `memory_items` row → always insert a new `watch_history` row for this viewing session → insert/update `timeline_events`.

**PDF Is Uploaded.** Upload bytes to Object Storage → insert `uploaded_files` → insert `memory_items` (`source_type='document'`) → insert `documents` (1:1, `storage_uri` from upload) → extract text per page → insert `document_pages` rows → enqueue `background_jobs` (`chunk_and_embed`).

**AI Conversation Is Imported.** Insert `memory_items` (`source_type='ai_conversation'`) → insert `ai_conversations` (1:1) → bulk insert `ai_messages` → enqueue `background_jobs` per message (or batched) to chunk + embed message content.

**Semantic Search Is Performed.** Insert `search_history` row → embed query text (not persisted as its own embeddings row — query embeddings are ephemeral) → pgvector similarity query against embeddings joined through `memory_chunks` → `memory_items` filtered by `user_id` → return ranked chunks with `source_locator` for citation display.

**Security Alert Is Generated.** (*Future*) Shield job evaluates footprints/privacy\_findings → inserts `security_alerts` → inserts notifications → `audit_logs` entry.

**Footprint Scan Runs.** (*Future*) `sync_jobs` row created (`connected_service_id` = the relevant OAuth connection) → job queries provider/breach-check sources → upserts footprints rows → may generate `privacy_findings` → `sync_jobs.status = 'completed'`.

---

## 16. Sample SQL Queries (Illustrative Only)

*These illustrate schema usage for developer understanding. They are not migration scripts.*

### Recent Memories

```
SELECT id, title, source_type, captured_at
FROM memory_items
WHERE user_id = :user_id AND deleted_at IS NULL
ORDER BY captured_at DESC
LIMIT 20;
```

### Timeline

```
SELECT te.event_at, te.event_type, mi.title, mi.source_type
FROM timeline_events te
JOIN memory_items mi ON mi.id = te.memory_item_id
WHERE te.user_id = :user_id
ORDER BY te.event_at DESC
LIMIT 50;
```

### Search History

```
SELECT query_text, result_count, searched_at
FROM search_history
WHERE user_id = :user_id
ORDER BY searched_at DESC
LIMIT 25;
```

### Most Visited Websites

```
SELECT ba.domain, COUNT(*) AS visits
FROM browser_activity ba
JOIN memory_items mi ON mi.id = ba.memory_item_id
WHERE mi.user_id = :user_id AND mi.deleted_at IS NULL
GROUP BY ba.domain
ORDER BY visits DESC
LIMIT 10;
```

### AI Conversations

```
SELECT ac.id, ac.title, ac.imported_from, COUNT(am.id) AS
message_count
FROM ai_conversations ac
LEFT JOIN ai_messages am ON am.ai_conversation_id = ac.id
WHERE ac.user_id = :user_id
GROUP BY ac.id
ORDER BY ac.created_at DESC;
```

### Security Alerts *(future schema)*

```
SELECT alert_type, severity, description, detected_at
FROM security_alerts
WHERE user_id = :user_id AND is_resolved = false
ORDER BY severity DESC, detected_at DESC;
```

### User Activity

```
SELECT action, COUNT(*) AS occurrences
FROM activity_logs
WHERE user_id = :user_id AND created_at > now() - interval '7 days'
GROUP BY action
ORDER BY occurrences DESC;
```

### Topic Search

```
SELECT mi.id, mi.title, t.name AS topic, mit.relevance_score
FROM memory_item_topics mit
```

```

JOIN topics t ON t.id = mit.topic_id
JOIN memory_items mi ON mi.id = mit.memory_item_id
WHERE t.user_id = :user_id AND t.name = :topic_name
ORDER BY mit.relevance_score DESC;

```

### Semantic Similarity Search (pgvector)

```

SELECT mc.id AS chunk_id, mc.content, mc.source_locator,
       mi.title, mi.source_type,
       e.vector <=> :query_vector AS distance
FROM embeddings e
JOIN memory_chunks mc ON mc.id = e.memory_chunk_id
JOIN memory_items mi ON mi.id = mc.memory_item_id
WHERE mi.user_id = :user_id AND mi.deleted_at IS NULL
ORDER BY e.vector <=> :query_vector
LIMIT 8;

```

---

## 17. Future Expansion

The schema is designed so the following additions are **additive**, not restructuring:

**17.1 Chrome / Edge / Firefox / Safari Extensions.** All extension clients talk to the same FastAPI/Postgres backend; no schema change required — `memory_items.source_type` and metadata already accommodate browser-origin differences (e.g., `metadata.browser = 'edge'`).

**17.2 Desktop App / Mobile App.** New client, same API surface. `sessions.device_label/user_agent` already generalize beyond “browser.”

**17.3 Slack / Discord Integration.** New `source_type` values ('slack', 'discord') on `memory_items`; a new lightweight subtype table per platform (mirroring articles/notes) if platform-specific fields are needed; `connected_services.service_type` gains new values; no change to core tables.

**17.4 Google Drive / Dropbox / OneDrive.** These map cleanly onto the existing `documents/uploaded_files` pattern — a new `storage_uri` scheme and a `connected_services` entry per provider; `documents` gains no new columns.

**17.5 GitHub.** Already named in MVP scope; a `repositories/commits` subtype pair can be added under Memory following the same supertype/subtype pattern as `documents/document_pages`.

**17.6 Notion.** Similar to Slack/Discord — a `notion_pages` subtype table plus a new `connected_services.service_type`.

**17.7 Calendar.** A `calendar_events` subtype table (`memory_item_id`, `starts_at`, `ends_at`, `attendees` JSONB) — fits the existing pattern exactly.

**17.8 Multi-Tenant Architecture.** The Section 6.6 placeholder tables (`organizations`, `team_workspaces`) become the tenant boundary. The migration path is additive: add a nullable `organization_id` to users (and optionally to `memory_items` for shared/team memory), defaulting to null for existing personal accounts — no existing row needs to change meaning.

**17.9 Enterprise Support.** `enterprise_settings` (Section 6.6) holds org-level policy overrides (mandatory SSO, stricter `never_capture_categories`, data-residency requirements) that layer on top of, rather than replace, the per-user `user_settings` already in place.

---

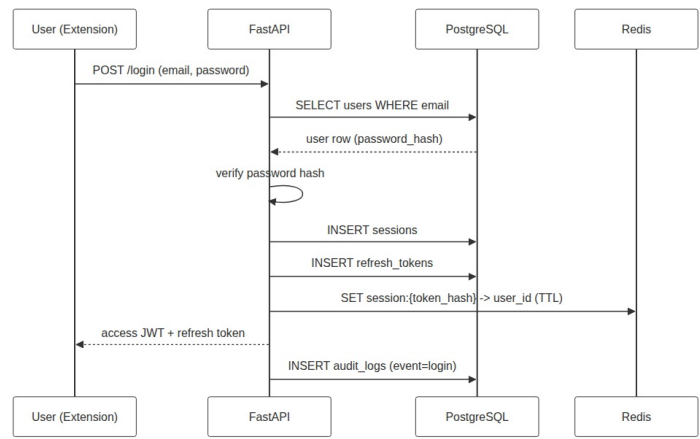
## 18. Best Practices

1. **Never bypass `memory_items` as the entry point.** Every new source type gets a `memory_items` row plus a subtype table — resist the temptation to model a “special” source as a standalone table outside this pattern, or Search/Timeline/RAG all need special-casing.
  2. **Treat `never_capture_categories` as a floor, not a default.** It must be enforced in application logic on every capture path, not just as a UI default a user could disable.
  3. **Every new table gets created\_at/updated\_at from day one**, even if it feels unnecessary at the time — retrofitting audit columns onto a live table with data is far more painful than including them upfront.
  4. **Soft-delete user content, hard-delete only via an explicit purge job.** Never let a single API call both soft-delete a row and immediately hard-delete its Object Storage file — keep those two operations decoupled and idempotent.
  5. **Every foreign key gets an index at creation time.** Do not wait for a slow-query report to discover a missing FK index.
  6. **Keep JSONB for genuinely variable shape only.** If a JSONB field’s keys turn out to be the same across 90% of rows, that’s a signal it wants to be a real column — revisit periodically rather than letting JSONB become a dumping ground.
  7. **Never store third-party secrets (OAuth tokens) unencrypted**, even in a development database — use the same encrypted-column pattern in every environment to avoid “it works in dev because we skip encryption there” bugs.
  8. **Keep Footprint/Shield reading from connected services, not re-implementing their own permission model.** This was a deliberate architecture decision (Section 6.4) and should not be re-litigated casually.
  9. **Review the index list quarterly.** Remove indexes that no query plan actually uses; add ones that `pg_stat_statements` shows are needed — index strategy should follow real query patterns, not assumptions made at design time.
  10. **Do not add multi-tenant columns speculatively.** Section 17.8’s migration path is intentionally deferred until team/enterprise features are actually being built — adding `organization_id` everywhere today would be premature complexity the PRD explicitly avoids.
- 

## 19. Sequence Diagrams — Key Workflows

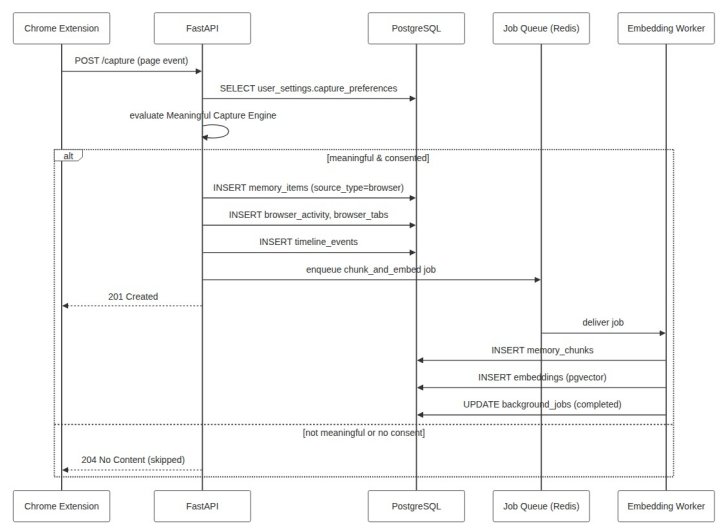
These complement Section 15 (Example User Flows) by showing the same workflows as actor-level interaction diagrams — useful when implementing the actual request/response and job-queue plumbing, rather than just the row-level effects.

### 19.1 Authentication



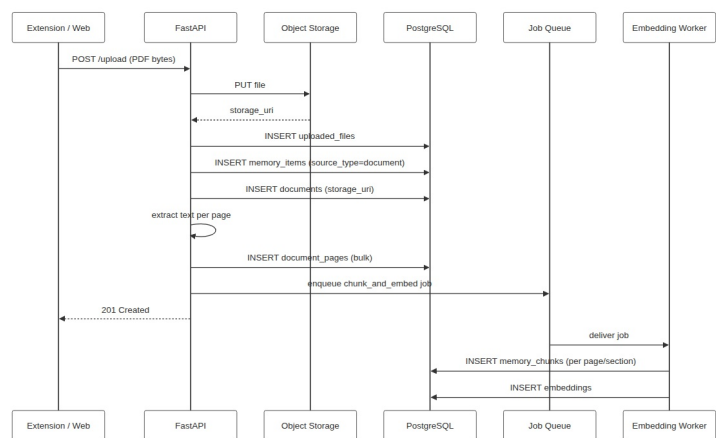
Authentication Sequence

## 19.2 Browser Capture



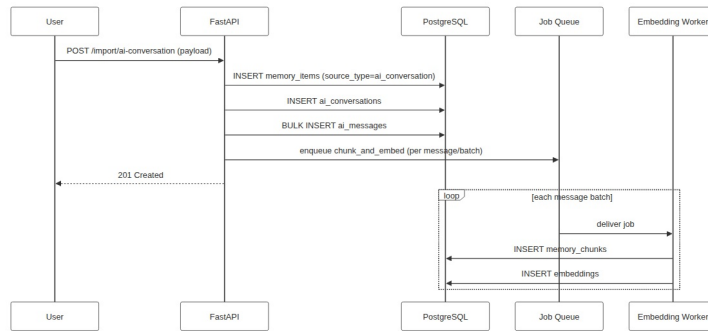
Browser Capture Sequence

## 19.3 PDF Upload



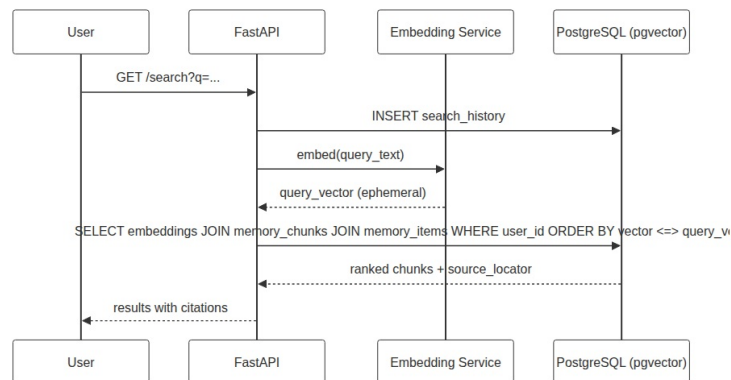
PDF Upload Sequence

## 19.4 AI Conversation Import



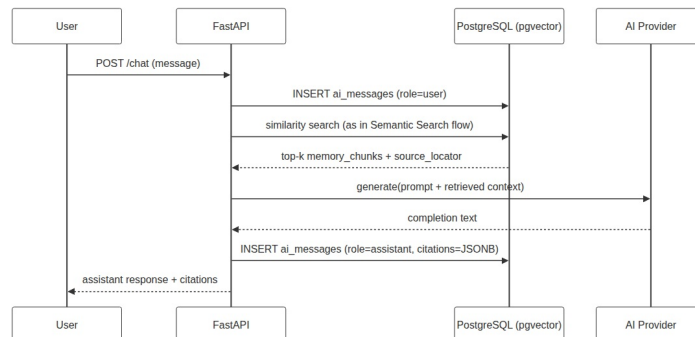
AI Conversation Import Sequence

## 19.5 Semantic Search



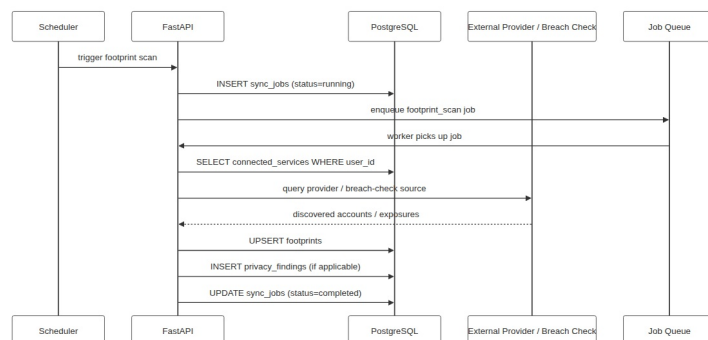
Semantic Search Sequence

## 19.6 RAG Response Generation



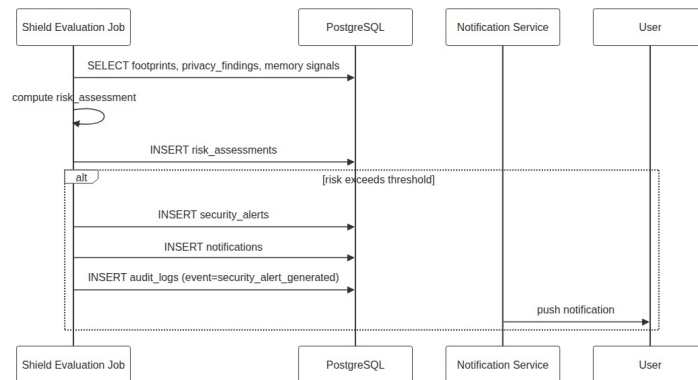
RAG Response Generation Sequence

## 19.7 Footprint Scan [FUTURE]



Footprint Scan Sequence

## 19.8 Security Alert Generation [FUTURE]

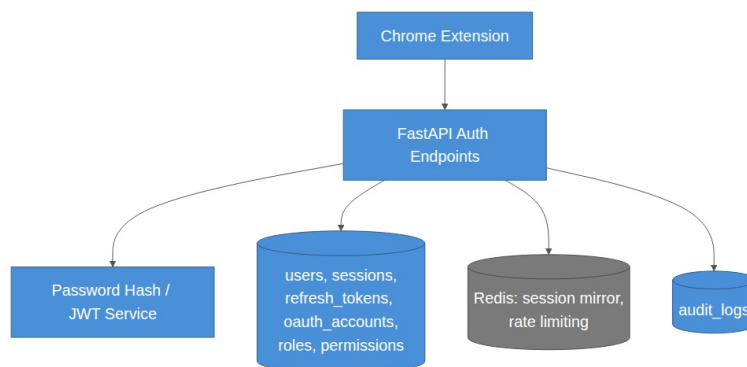


Security Alert Generation Sequence

## 20. Module-Level Architecture Diagrams

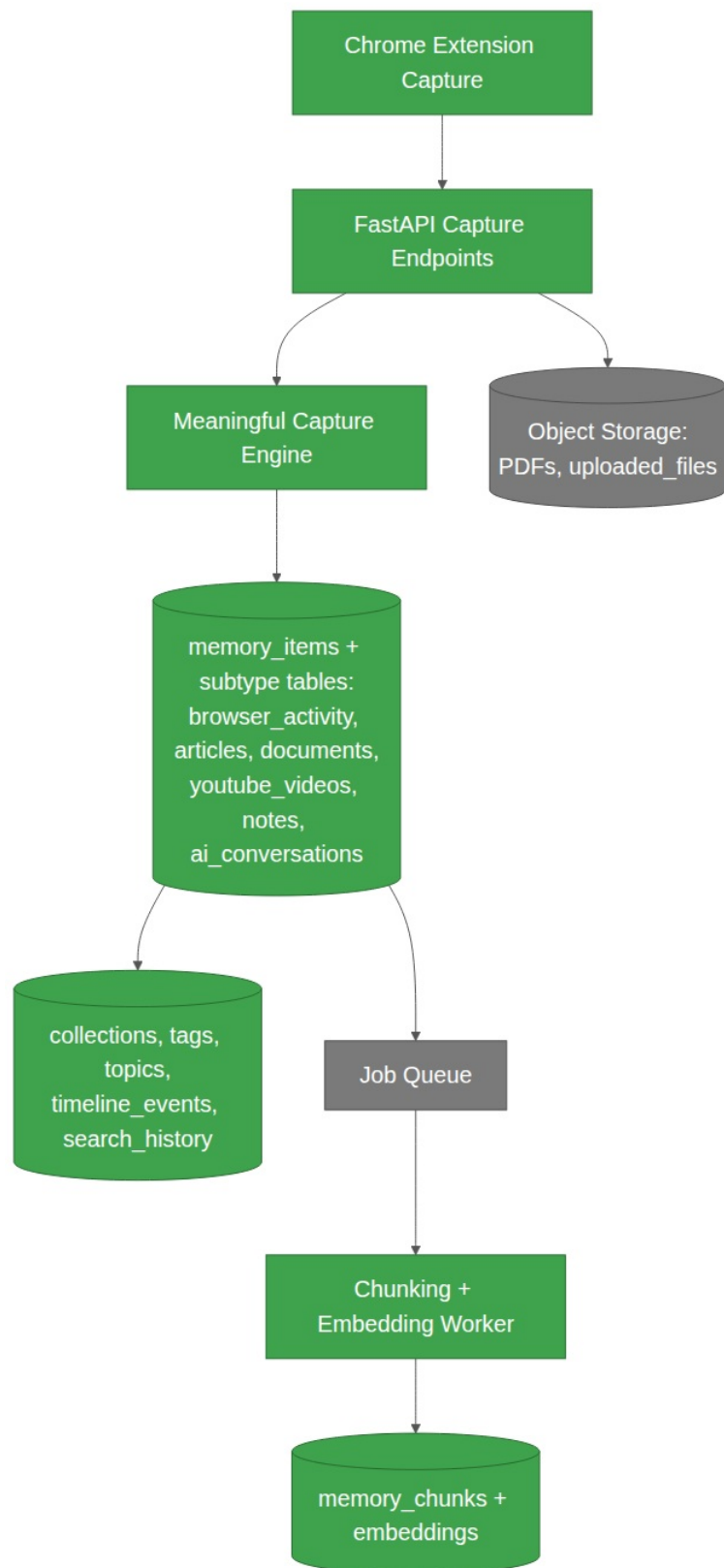
Where Section 5 shows *data* relationships and Section 19 shows *request-level* interactions, these diagrams show each module's internal component layering — the services and stores a backend engineer would actually build.

### 20.1 Authentication Architecture



Authentication Architecture

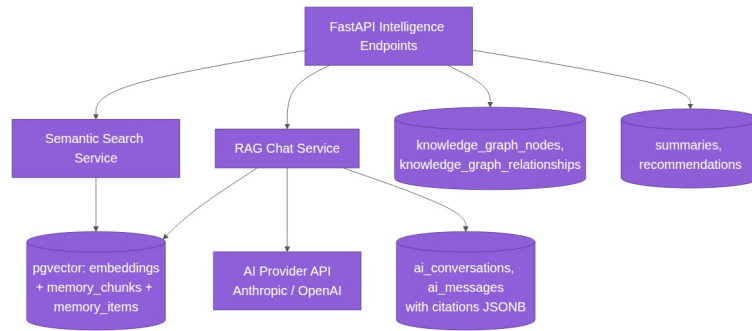
### 20.2 Memory Architecture



Memory Architecture

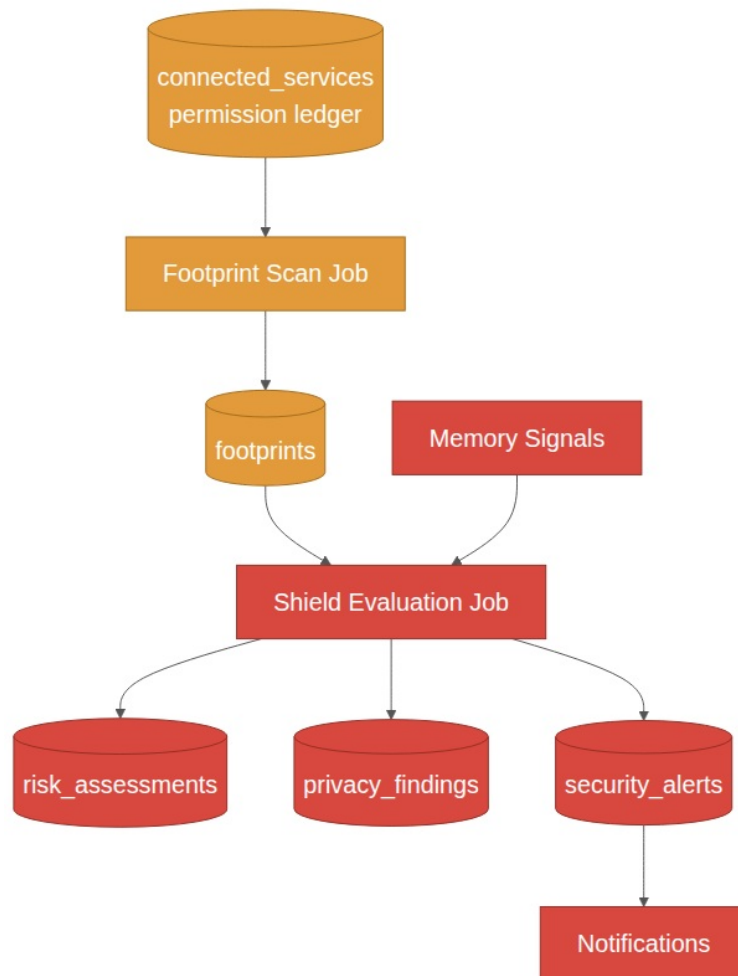
## 20.3 Intelligence Architecture





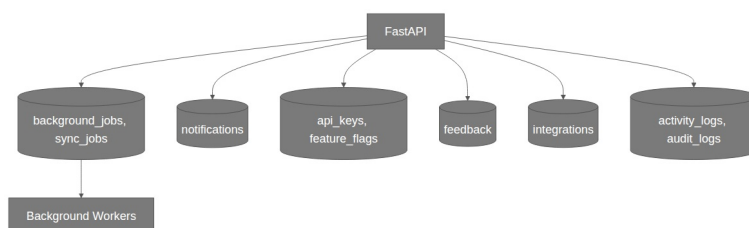
Intelligence Architecture

## 20.4 Footprint & Shield Architecture [FUTURE]



Footprint and Shield Architecture

## 20.5 System Architecture



System Architecture

## 21. One-Page Schema Summary

A single reference view for onboarding — module scope, table count, and what connects to what, without needing to read Sections 5–6 in full.

| Module              | Status     | Table Count | Core Root Table  | Connects To                                                      |
|---------------------|------------|-------------|------------------|------------------------------------------------------------------|
| Authentication      | MVP        | 9           | users            | Every other module (owns user_id FK)                             |
| Memory              | MVP        | 20          | memory_items     | Intelligence (via memory_chunks), Shield (signal source)         |
| Intelligence (AI)   | MVP + V1.x | 8           | embeddings       | Memory (memory_chunks), Intelligence (owns ai_conversations)     |
| Footprint           | Future     | 1           | footprints       | Authentication (connected_services feeds Shield)                 |
| Shield              | Future     | 3           | risk_assessments | Footprint (input), Memory (signal input), System (notifications) |
| System              | MVP + V1.x | 10          | background_jobs  | Cross-cutting — referenced by every module's async operations    |
| Future Placeholders | Future     | 6           | organizations    | Attaches to Authentication (users.organization_id not yet added) |

**Total schema size (MVP + near-term V1.x, excluding Future placeholders): ~47 tables across 6 modules.**

## 22. Table Dependency Map

This shows creation/foreign-key order — the sequence in which tables must exist before their dependents can be migrated. Split into two diagrams (core auth/memory vs. AI/security/system) to stay legible on an A4 page.

### 22.1 Authentication Dependency Chain

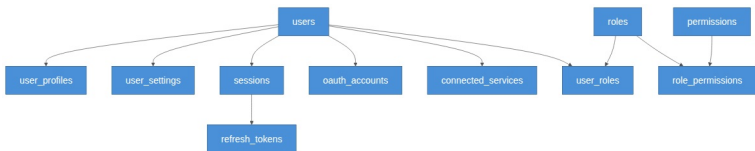


Table Dependency Map - Authentication

### 22.2 Memory Capture Dependency Chain

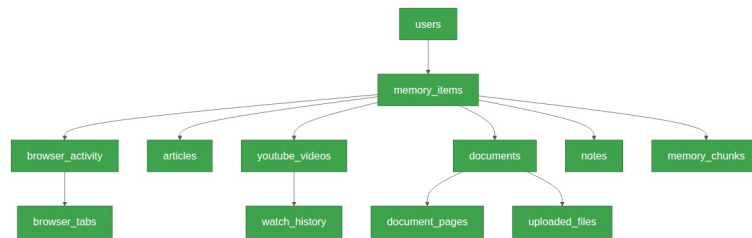


Table Dependency Map - Memory Capture

## 22.3 Intelligence Dependency Chain

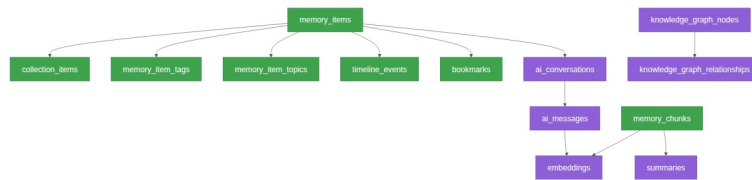


Table Dependency Map - Intelligence

## 22.4 Footprint, Shield & System Dependency Chain

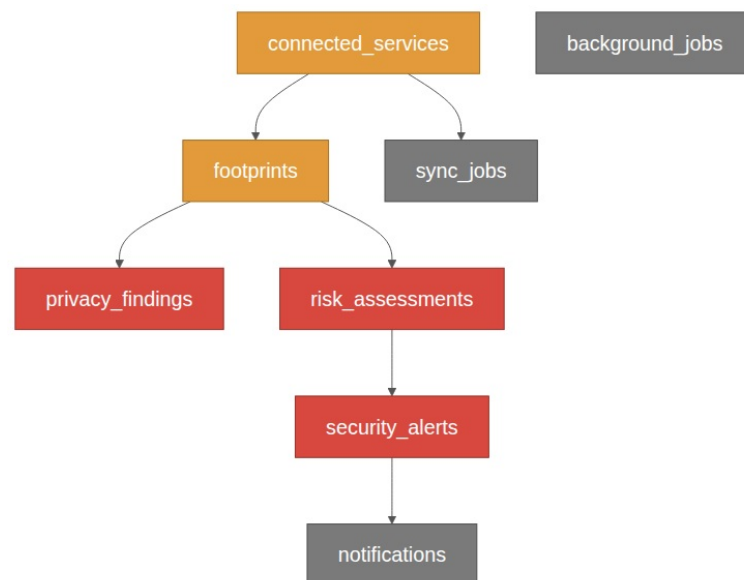


Table Dependency Map - Footprint Shield System

**How to read this for migration ordering:** an arrow from A to B means B's table must be created after A's (B holds a foreign key to A). Both diagrams flow top-to-bottom in the order migrations should run within their group; users and memory\_items are the two tables nearly everything downstream ultimately depends on, directly or transitively.

## 23. Naming Conventions Cheat Sheet

A condensed, at-a-glance version of the fuller rationale in Section 4.

| Element     | Convention           | Ex:               |
|-------------|----------------------|-------------------|
| Table name  | snake_case, plural   | memory_items, sec |
| Column name | snake_case, singular | user_id, captured |
| Primary key | Always id, type UUID | id                |

|                             |                                                                     |                             |
|-----------------------------|---------------------------------------------------------------------|-----------------------------|
| Foreign key                 | <referenced_table_singular>_id                                      | user_id, memory_i           |
| Boolean column              | Prefixed is_ / has_                                                 | is_active, has_co           |
| Timestamp column            | Suffixed _at, always TIMESTAMPTZ                                    | created_at, captu           |
| Soft delete marker          | deleted_at, nullable                                                | deleted_at IS NU            |
| Bridge (many-to-many) table | <table_a_singular>_<table_b_singular>                               | memory_item_tags            |
| JSONB metadata column       | Named metadata, payload, or config — never bare data                | metadata, capture           |
| Enum-like status column     | Named status, plain TEXT with a documented value set, not a DB enum | status = 'active' 'expired' |
| Index naming                | ix_<table>_<column(s)>                                              | ix_memory_items_            |
| Vector column               | Always named vector, type VECTOR(n)                                 | embeddings.vecto            |

## 24. Diagram Legend Reference

Every diagram in this document (ERDs, architecture diagrams, dependency maps) uses one consistent color per module. Sequence diagrams (Section 19) are left uncolored since they are actor/message flows, not entity ownership views.

Authentication

Memory

Intelligence / AI

Footprint - future

Shield - future

System

Diagram Color Legend

| Color  | Module             | Applies To                                               |
|--------|--------------------|----------------------------------------------------------|
| Blue   | Authentication     | users, sessions, roles, permissions, etc.                |
| Green  | Memory             | memory_items and all capture/organization subtype tables |
| Purple | Intelligence (AI)  | embeddings, ai_conversations, knowledge graph, summaries |
| Orange | Footprint [FUTURE] | footprints, connected_services (shared with Auth)        |
| Red    | Shield [FUTURE]    | security_alerts, privacy_findings, risk_assessments      |
| Gray   | System             | background_jobs, audit_logs, notifications, etc.         |

25. Recommended Backend Folder Structure

A conventional FastAPI + SQLAlchemy + pgvector layout that mirrors this document's module boundaries — models, migrations, repositories, and vector-specific components are kept separate so the Memory/Intelligence split in the schema is reflected in the codebase, not just the database.

```

backend/
├── app/
│   ├── main.py
│   ├── core/
│   │   ├── config.py
│   │   └── security.py          # JWT, password hashing, RBAC
├── checks
│   └── redis_client.py
├── models/                      # SQLAlchemy ORM models, one file
│   └── per module
│       ├── auth/
│       │   ├── user.py
│       │   ├── session.py
│       │   ├── oauth_account.py
│       │   └── role_permission.py
│       ├── memory/
│       │   ├── memory_item.py
│       │   ├── browser.py      # browser_activity, browser_tabs
│       │   └── document.py     # documents, document_pages,
├── uploaded_files
│   ├── youtube.py
│   ├── note.py
│   ├── collection.py
│   ├── tag_topic.py
│   ├── intelligence/
│   │   ├── ai_conversation.py
│   │   └── embedding.py       # pgvector column definition
├── lives here
│   ├── knowledge_graph.py
│   ├── summary_recommendation.py
│   ├── security/              # Footprint + Shield (future)
│   │   ├── footprint.py
│   │   └── shield.py
│   ├── system/
│   │   ├── job.py             # background_jobs, sync_jobs
│   │   ├── notification.py
│   │   └── audit_log.py
├── repositories/              # Data-access layer, one per
├── module, mirrors models/
│   ├── auth_repository.py
│   ├── memory_repository.py
│   ├── intelligence_repository.py
│   ├── security_repository.py
│   └── system_repository.py
├── vector/                    # pgvector-specific components,
├── isolated from generic repos
│   ├── chunking.py            # text chunking strategy
│   └── embedding_client.py    # calls out to embedding
├── model/provider
│   ├── similarity_search.py   # builds/executes <=> queries
│   └── citation_builder.py    # assembles source_locator-based
├── citations
│   └── services/              # Business logic orchestration
├── (calls repositories + vector/)
│   ├── auth_service.py
│   └── capture_service.py     # Meaningful Capture Engine
└── lives here

```

```

├── search_service.py
├── rag_chat_service.py
├── footprint_shield_service.py    # future
├── api/                          # FastAPI routers, thin –
├── delegate to services/
│   ├── v1/
│   │   ├── auth.py
│   │   ├── memory.py
│   │   ├── intelligence.py
│   │   └── system.py
│   ├── jobs/                    # Background worker task
│   └── definitions
│       ├── chunk_and_embed.py
│       ├── footprint_scan.py    # future
│       └── shield_evaluation.py # future
│   └── schemas/                # Pydantic request/response
├── models, mirrors models/
│   ├── auth.py
│   ├── memory.py
│   ├── intelligence.py
│   └── system.py
├── migrations/                  # Alembic migrations – one
├── linear history
│   ├── versions/
│   │   └── <timestamp>_<description>.py
│   └── env.py
├── seeds/                       # Idempotent seed scripts
├── (roles, permissions, ai_providers, feature_flags)
│   └── seed_defaults.py
├── tests/
│   ├── models/
│   ├── repositories/
│   └── services/

```

**Why this shape.** `models/` and `repositories/` both mirror the six-module split used throughout this document, so a developer who has read Section 6 already knows where to find (or add) code. `vector/` is deliberately pulled out of `repositories/` — `pgvector` query construction, chunking, and citation logic are a distinct concern from ordinary CRUD and tend to change independently (e.g., swapping embedding models per Section 9.6) without touching relational repository code.

## 26. Known Limitations and Trade-offs

Documented deliberately rather than discovered in production — every current constraint below is a conscious MVP trade-off, not an oversight.

| Limitation                                | Why It's Accepted at MVP                                     | Migration Path When Outgrown                                                                              |
|-------------------------------------------|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Single Postgres instance, no read replica | MVP write/read volume doesn't justify replica-lag complexity | Add a read replica for analytics/reporting queries first; app queries stay primary-only until proven safe |
|                                           | Row counts                                                   | <code>activity_logs/audit_logs</code>                                                                     |

|                                                            |                                                                                     |                                                                                                                           |
|------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| No table partitioning                                      | (memory_items, activity_logs) are small pre-scale                                   | partition by month; memory_chunks/embeddings partition by user_id hash if multi-tenant                                    |
| pgvector instead of a dedicated vector DB                  | Per-user corpus size is small; avoids dual-write consistency risk                   | Migrate to a dedicated vector DB only if a single user's embedding count reaches the tens-of-millions range (Section 2.2) |
| embeddings.vector fixed at one dimension                   | Only one embedding model in production at a time                                    | New model with a different dimension requires an explicit embeddings_v2 migration (Section 9.6), not a silent ALTER       |
| RBAC has only two roles (user, admin)                      | No team/workspace concept yet                                                       | Table structure (Section 6.1) already supports more roles — no redesign needed, just new seed data                        |
| No multi-tenant/organization_id columns                    | Personal-use product, no team billing yet                                           | Section 17.8's additive migration path (nullable organization_id)                                                         |
| Soft-delete grace period is a single global constant       | One retention policy is enough for one user tier                                    | enterprise_settings (Section 6.6) becomes the per-org override point                                                      |
| Object Storage has no built-in versioning exposed to users | Captured files are immutable snapshots at MVP                                       | Enable provider-level versioning as a user-facing "file history" feature post-MVP                                         |
| Query embeddings are ephemeral, not cached                 | Search query variety is high; hit rate on repeat identical queries is low pre-scale | Add a short-TTL Redis cache keyed by normalized query text if repeat-query analytics justify it                           |

## 27. Performance Targets

Latency goals for the operations that matter most to perceived product speed. These are **targets to design against**, not measured SLAs — they inform indexing and caching decisions made earlier in this document (Sections 8, 11).

| Operation                           | Target (p95) | Primary Lever                                                           |
|-------------------------------------|--------------|-------------------------------------------------------------------------|
| Login / JWT issuance                | < 200 ms     | Indexed users.email lookup + bcrypt/argon2 cost factor tuning           |
| Session validation (per request)    | < 15 ms      | Redis-mirrored session lookup (Section 11.2), not a Postgres round-trip |
| Memory capture write (browser/note) | < 300 ms     | Single-row inserts on already-indexed tables; embedding happens async   |
| Timeline page load (20 items)       | < 150 ms     | Composite index memory_items(user_id, captured_at DESC)                 |



|                                                              |                                         |                                                                                                                        |
|--------------------------------------------------------------|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Semantic search<br>(top-8 results)                           | < 600 ms                                | (Section 8)<br>HNSW index on<br>embeddings.vector;<br>query embedding call<br>is the dominant cost                     |
| RAG chat response<br>(first token)                           | < 2.5 s                                 | Retrieval (~same as<br>search) + LLM<br>provider latency;<br>retrieval must stay a<br>small fraction of this<br>budget |
| PDF upload + text<br>extraction (10 pages)                   | < 5 s                                   | Async: user gets 201<br>immediately,<br>extraction/chunking<br>happens in the<br>background job                        |
| Chunk + embed a<br>captured item                             | < 10 s (background,<br>not user-facing) | Job queue<br>throughput (Section<br>11.4), not request<br>latency                                                      |
| Footprint scan (per<br>connected service)<br><i>(future)</i> | < 30 s (background)                     | External provider API<br>latency dominates;<br>not Postgres-bound                                                      |

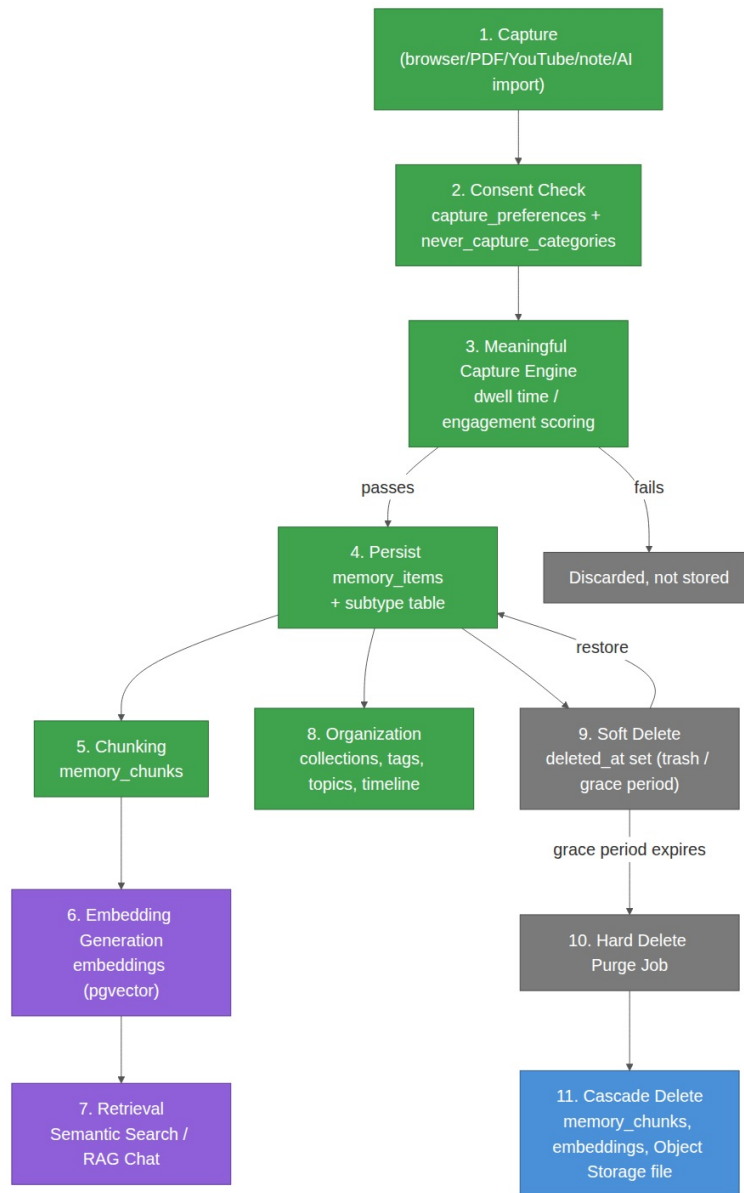
---

**\* Developer Note.** If semantic search regularly exceeds target, check the WHERE mi.user\_id = ... filter is applied *before* the ORDER BY <=> (Section 9, Developer Note) — a missing or late filter forces pgvector to rank far more rows than necessary.

---

## 28. Memory Lifecycle — Capture Through Deletion

A single end-to-end view of a memory item's life, tying together capture (Section 15), retrieval (Sections 16, 19.5–19.6), and deletion (Section 7's cascade rules, Section 12.8's retention policy) — the workflow diagrams elsewhere each show one slice of this.



Memory Lifecycle

### Key transitions:

- 1. Capture → Consent Check → Meaningful Capture Engine.** Nothing reaches `memory_items` without clearing both the user's `capture_preferences` and the meaningfulness threshold — items that fail either gate are never persisted (not soft-deleted; they simply never existed as a row).
- 2. Persist → Chunk → Embed.** Persistence is synchronous (part of the API response); chunking and embedding are always asynchronous background work, matching the Performance Targets above.
- 3. Organization is additive, not sequential.** Collections/tags/topics can be applied at any point after persistence, not just at capture time — the diagram shows this as a parallel branch, not a further pipeline stage.
- 4. Soft Delete is the default deletion path.** `deleted_at` is set; the item can still be restored. Only the scheduled purge job (Section 12.8) converts this into a hard delete.
- 5. Hard Delete cascades everywhere.** Postgres FK CASCADE removes `memory_chunks`/embeddings; a separate async step removes the Object Storage file (Section 10) — deliberately decoupled so a slow storage call never blocks the user-facing delete confirmation.

## 29. Common Query Patterns

The query patterns Sentiora’s backend runs constantly — documented once here with their join shape, index usage, cache strategy, and complexity, so new endpoints reuse a known-good pattern instead of re-deriving one. (Section 16 has full SQL text for several of these; this section is the “which pattern, when, and how expensive” reference.)

| Pattern                                         | Typical Join Path                                                    | Index Used                                                    |
|-------------------------------------------------|----------------------------------------------------------------------|---------------------------------------------------------------|
| <b>Timeline feed</b>                            | timeline_events → memory_items                                       | memory_items(user_id, captured_at DESC)                       |
| <b>Filtered browse</b><br>(“show my PDFs”)      | memory_items (filtered)                                              | memory_items(user_id, source_type)                            |
| <b>Semantic search</b>                          | embeddings → memory_chunks → memory_items                            | HNSW on embeddings.vector<br>filtered by user_id post-join    |
| <b>RAG chat turn</b>                            | Same as semantic search, plus ai_conversations → ai_messages         | Same HNSW index + ai_messages(ai_conversation_id, created_at) |
| <b>Collection view</b>                          | collection_items → memory_items                                      | FK index on collection_items(collection_id)                   |
| <b>Tag/topic filter</b>                         | memory_item_tags → memory_items or memory_item_topics → memory_items | FK index on the bridge table                                  |
| <b>Session validation</b>                       | None — Redis only, Postgres is fallback                              | sessions(user_id, is_active) (fallback path only)             |
| <b>Audit trail lookup</b><br>(admin/compliance) | audit_logs (filtered by user_id, event_type)                         | Composite index on (user_id, created_at)                      |
| <b>Dependency-ordered migration check</b>       | N/A — schema introspection, not runtime                              | N/A                                                           |

✳ **Developer Note.** A new endpoint that doesn’t fit one of these patterns is a signal to pause and check: does it need a new index (Section 8), or is it accidentally about to do an unfiltered scan over memory\_chunks/embeddings — the two tables most likely to grow unbounded (Section 13)?